

Free Range Routing Monitoring and Anomaly Detection Tool

Bachelor's Thesis
Documentation

Semester: Spring 2025



Version: 01.00

Date: 2025-06-23

Project Team: Yannick Staedeli
Mino Petrizzo
Roman Cvijanovic

Advisor: Severin Dellsperger

Internal Co-Examiner: Olaf Zimmermann

External Co-Examiner: Thomas Graf

Project Partner: Open-Systems - Julian Klaiber



School of Computer Science
OST Eastern Switzerland University of Applied Sciences

Abstract

Modern dynamic routing environments often suffer from limited visibility and diagnostic capabilities at the individual router level. This is especially true for setups based on Free Range Routing (FRR), an open-source routing suite that implements routing protocols like Open Shortest Path First (OSPF) and Border Gateway Protocol (BGP). Operators are typically forced to inspect each router's state manually via Command Line Interface (CLI), making it difficult to detect and understand anomalies such as missing or unexpected route advertisements. These issues can compromise network stability and make troubleshooting time-consuming and error-prone.

We developed FRR-MAD, a modular, open-source monitoring and anomaly detection tool tailored to OSPF in FRRouting, to address these issues. It parses and analyzes routing data, detects inconsistencies, and visualizes the router's state through an interactive text-based user interface (TUI). FRR-MAD consists of two components: the *frr-mad-analyzer*, which acts as the backend, collecting and evaluating OSPF data from FRR and the *frr-mad-tui*, a terminal-based frontend that presents the data for monitoring and troubleshooting. Features like structured route overviews, live anomaly highlighting, filtering, and Prometheus-compatible export simplify OSPF diagnostics and make routing behavior easier to understand, even in complex environments. An additional artifact that arose from this thesis is a containerlab-based development and test environment, which allows users to quickly spin up a realistic multi-router topology and test FRR-MAD without impacting production systems.

FRR-MAD enhances the monitoring and troubleshooting of OSPF routing in FRR environments by providing live anomaly detection and intuitive visualization. Its open-source nature encourages community collaboration. We invite the FRR community to adopt, test, and contribute to the project¹. Developed in close collaboration with our project partner Open Systems, this tool is now being integrated into their production infrastructure.

¹<https://github.com/FRR-MAD/frr-mad>

Management Summary

Management summary outlines the project's objective, methodology, result and achievements. Compared to the abstract it provides more clarity and detail what was explicitly achieved.

Current State

Free Range Routing (FRR) is a widely used network routing software suite. It implements a variety of different routing protocols like Open Shortest Path First (OSPF). However, the implementations are occasionally suboptimal due to software faults, which can lead to unexpected behavior in practice. As a result, the implementation of OSPF in FRR behaves sometimes inconsistently. This behavior can be seen by checking the Link State Database (LSDB) against FRR's static file configuration. The inconsistency appears as wrongly advertised Link-State Database (LSDB) entry. This issue is made worse by the fact, that there are no available open source solution to report such inconsistent behavior. To battle this behavior users of FRR are forced to create custom monitoring and observability processes to provide high availability of their services. And even if this inconsistency is spotted, it still needs to be manually verified. This can only be done on the device presenting the issue and only by using the interactive command terminal of FRR. This also highlights another issue. This issue requires an operator of such a service to directly interactive with a tool that can apply runtime changes. Advanced network engineers will have no issues correctly handling vtysh, but the same cannot be claimed for entry or medium level engineers. Therefore a monitoring solution to observe the current OSPF state is required.

Methodology

The project followed an agile development methodology based on the SCRUM framework. In close collaboration with the industry partner, multiple meetings were held to define and iteratively refine the Functional Requirements. This helped prioritize key features for development. In addition, relevant

Non-Functional Requirements were established to ensure high product quality.

To establish a solid foundation, a two-sprint elaboration phase was conducted. The goal of this phase was to evaluate existing tools and technologies that could be integrated into the project, thereby reducing development effort and focusing on value-adding components.

The subsequent development phase was the most substantial part of the project. All features were implemented and tested during this time. As the core components took shape, some initial requirements had to be revised, and previously considered integration solutions were discarded in favor of custom implementations.

The final solution comprises two distinct components: the `frr-mad-analyzer`, a background daemon responsible for data collection and analysis, and the `frr-mad-tui`, a terminal-based user interface that enables real-time interaction with the analyzer.

Achievements

The results of this bachelor's thesis are the previously mentioned `frr-mad-analyzer` and `frr-mad-tui`.

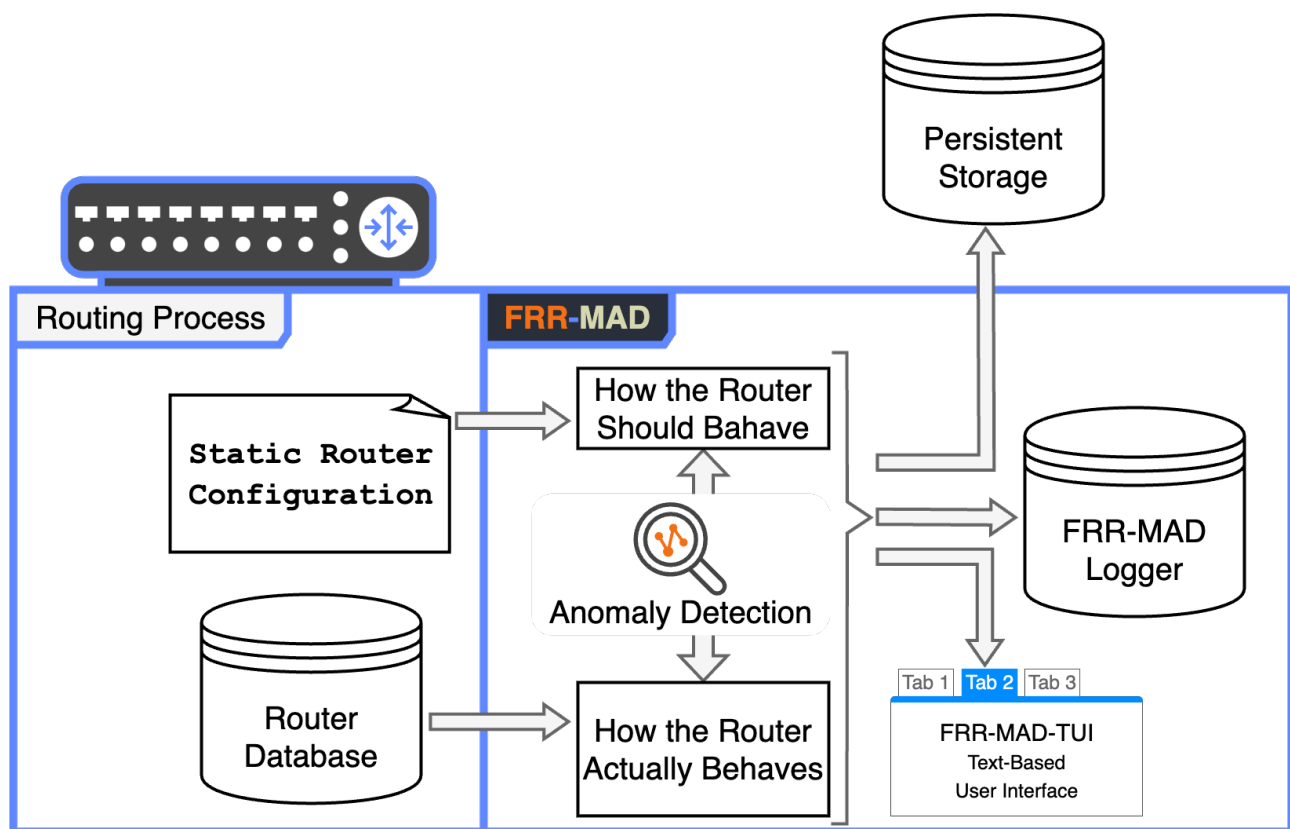


Figure 1: Simplified Architecture

The `frr-mad-analyzer` runs quietly in the background on a server. As long as the server is using Free Range Routing (FRR) and has routing enabled, it regularly checks two types of routing information: what the routing should look like, and what it actually looks like. It uses this comparison to find and report any problems or unexpected behavior in the network.

The frr-mad-analyzer works by creating two separate views of the network. The first view is based on the configuration and shows how the routing is expected to behave. The second view is based on the current state of the router and shows how the routing is actually working. By comparing these two views, it can quickly detect anything unusual or incorrect. If a route is found in one view but not the other, it is marked as a problem. Each issue is clearly described to help understand what is wrong and why.

In addition to finding problems, the frr-mad-analyzer also replaces an older monitoring tool used by the project partner. For this reason, it also includes built-in support to send status and performance data to external monitoring systems.

The second main part of the system is the frr-mad-tui. This is a simple, text-based interface that allows users to interact with the analyzer. It shows a clear summary of the router's current status and any problems that have been found. It also lets users explore detailed routing information in an easy-to-read format. This helps people who are not entry-level network engineers quickly understand how the router is performing.

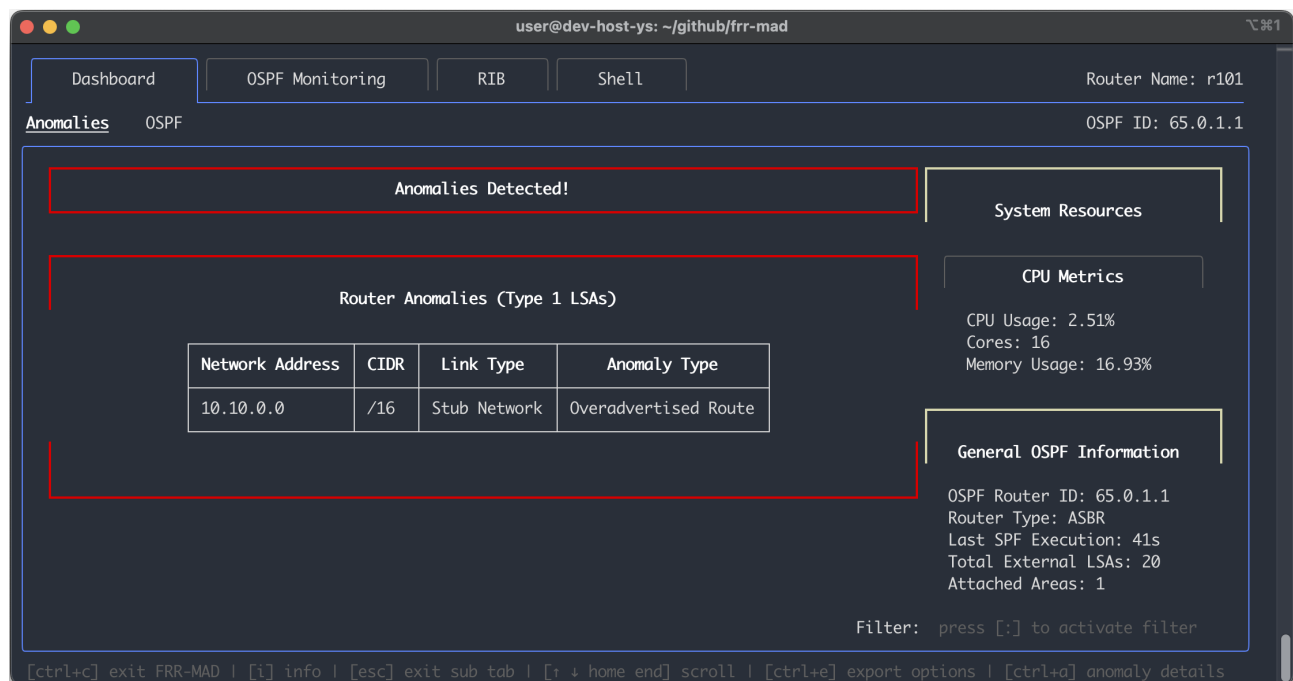


Figure 2: frr-mad-tui Anomaly Dashboard

Future Work

Both project components were released with version 1.0.0 on [GitHub](#). But this only contains the features outlined in the requirements. During the development many experiences were gathered and some compromises were made to meet industry partner's expectations. Following that, the project will be open sourced with the organization name Free Range Routing - Monitoring and Anomaly Detection. The short term plans are:

- Adjust the anomaly detection to be compatible with the newest version of Free Range Routing.
- Enable increased modularity for the daemon component to support more standalone features. For example, a software running in the background that only provides basic monitoring data for external tools.
- Adjustment of the parsing system to support different configuration styles.

After implementing all pending shortcomings, the project will effectively enter maintenance mode. It will accept feature requests and bug reports as per the norm.

Contents

I	Glossary and Acronyms	viii
II	Technical Report	1
1	Introduction	2
1.1	General	2
1.1.1	Technical Report	2
1.1.2	Project Documentation	3
1.2	Terms & Techniques	3
1.2.1	FRR-MAD	3
1.2.2	Free Range Routing	4
1.2.3	Open Shortest Path First	4
1.3	Aims and Objectives	4
1.3.1	Problem	4
1.3.2	Solution	7
2	Results	10
2.1	Project Boundaries	10
2.2	Project Achievements	11
2.2.1	Gather FRR Route Information	11
2.2.2	Persistently store FRR Route Information	12
2.2.3	Visualize FRR Route Information	13
2.2.4	Export Parsed FRR Route Information	16
2.2.5	OSPF Anomaly Detection	18
2.2.6	Export OSPF Anomalies	19
2.2.7	Visualize Detected Anomalies	20

2.2.8	Other TUI Features	20
2.2.9	Augmented Development and Testing Environment	22
2.3	Implementation	23
2.3.1	Architecture	24
2.3.2	Backend Service	28
2.3.3	Frontend Client	34
2.3.4	Process and Challenges	35
3	Conclusion	42
3.1	Outcome Analysis	42
3.1.1	Functional Requirements Evaluation	43
3.1.2	Non-Functional Requirements Evaluation	50
3.2	Future Directions	51
3.2.1	Protocol Support Extensions	51
3.2.2	Anomaly Detection Enhancements	51
3.2.3	User Experience Improvements	52
3.2.4	Community and Operational Features	52
	Bibliography	52
	List of Figures	55
	List of Tables	56
	List of Algorithms	57
	List of Code Blocks	58
A	Miscellaneous	60
A.1	Industry Partner's Anomaly Occurrence	60
A.2	Development and Test Environment Architecture	62
A.3	Setup and Installation	64
A.3.1	Build Instructions	64
A.3.2	Application Launch	66

Part I

Glossary and Acronyms

General Terms

aggregator The **aggregator** is a component of the frr-mad-analyzer service. It is responsible for querying vtysh correctly and parsing the received information into structured data objects. 3, 7, 28

analyzer The **analyzer** is a component of the frr-mad-analyzer service. The analyzer receives data objects from the aggregator and analyzes it for anomalous behavior. The results are stored in structured data objects and provided to the exporter and socket components. 3, 7

Batfish Batfish is an open-source tool that analyzes network configuration files to detect potential issues like routing loops, unreachable networks, and policy violations before deployment. It builds a model of your network from vendor configs (Cisco, Juniper, etc.) and runs verification queries to catch configuration errors in a safe, offline environment. 37

Cobra Configuration File The standardized YAML configuration file format used by the Cobra CLI framework, which serves as the central configuration mechanism for the entire FRR-MAD system. 13

containerlab A tool for orchestrating container-based networking labs, enabling easy deployment of virtual network topologies for testing and development purposes. 9

daemon A background process that runs continuously to perform system or network tasks. iii

Docker A platform for developing, shipping, and running applications in lightweight containers. It allows consistent environments across development, testing, and production systems. 48

exporter The **exporter** is a component of the frr-mad-analyzer service. Its sole responsibility lies in exporting received information from the aggregator and analyzer components. 3, 7, 25, 28

Free Range Routing (FRR) Free Range Routing is an open source routing stack that provides a suite of tools for managing IP routing. It is designed to be highly customizable and can be used to build a wide range of IP routing applications. ii, 4

Free Range Routing - Monitoring and Anomaly Detection (FRR-MAD) The Project name of this bachelor's thesis. 3, 7, 23

FRR-MAD TUI Write Mode A privileged mode within the frr-mad-tui, activated using `Ctrl+W`, that grants the user access to execute shell commands and any vtysh command. Since this mode permits configuration changes, a safety mechanism ensures that the application can only be exited if the running-config matches the startup-config. 8, 14

frr-mad-analyzer The backend component of FRR-MAD that collects, analyzes, and exposes OSPF routing data and anomalies. iii, iv, 3, 7

frr-mad-tui A Text-Based User Interface (TUI) for visualizing OSPF routing data and anomalies detected by the frr-mad-analyzer. iii, iv, 3, 8, 13, 20, 21, 34, 47, 49

iTerm2 A terminal emulator for macOS that offers technical features such as split panes, inline images, and support for OSC52 clipboard integration. 48

Label Cardinality The number of unique label combinations in a Prometheus time series. High cardinality increases memory usage and can degrade query performance. 12

OSC52 Protocol A terminal escape sequence that allows copying text to the system clipboard over SSH by encoding the data in the terminal output stream. 17, 48

socket The **socket** is a component of the frr-mad-analyzer service. It ties together the frontend and the backend. Spawning a Unix socket, it enables the frontend direct access to the various gathered and analyzed metrics. 3, 8

Technical Concepts

AS External LSA (Type 5 LSA) AS-external-LSAs describe routes to destinations external to the Autonomous System. AS-external-LSAs are the only type of LSAs that are flooded throughout the entire Autonomous System (except stub areas). For a comprehensive overview and deeper insights, visit [RFC2328, Chapter AS-external-LSAs](#). 12, 13, 18, 41, 45, 60

ASBR Summary LSA (Type 4 LSA) ASBR-summary-LSAs are generated by ABRs to describe routes to Autonomous System Boundary Routers (ASBRs). These LSAs allow routers within an area to reach inter-AS destinations. See [RFC2328](#) for more information. 13

Duplicated Route A route that is announced (advertised) multiple times with conflicting attributes, such as different costs or paths. This can cause confusion in the network, as devices may not know which route to use, potentially leading to inefficient or incorrect routing of data. 45

Forwarding Information Base (FIB) A lookup table used by routers to actually forward packets. It contains only the best route to each destination, selected from the Routing Information Base (RIB). 6, 14, 16

Free Range Routing - Monitoring and Anomaly Detection An open-source monitoring and issue detection toolkit for FRRouting. It consists of a daemon for static frr config and lsdB cross analysis and anomaly detection. Reporting useful information via a self-built tui or a prometheus node exporter compatible layer. iv

frr_exporter Prometheus exporter for FRR version 3.0+ that collects metrics from the FRR Unix sockets and exposes them via HTTP, ready for collecting by Prometheus. 4, 37, 38

Interface List In OSPF context, this refers to the collection of network interfaces participating in OSPF routing, including their states, assigned areas, network types, and associated metrics. 13

Interior Gateway Protocol (IGP) A class of routing protocols used to exchange routing information within a single autonomous system. Common IGPs include OSPF, RIP, and IS-IS. 4

Link State Database (LSDB) A structured repository maintained by link-state routing protocols, such as OSPF, containing detailed topology information about all network segments and their interconnections. Routers within an OSPF area synchronize their LSDBs to ensure consistent and accurate routing decisions throughout the network. ii, 11, 13

Link State Routing Algorithm An interior routing protocol that ensures each router maintains accurate knowledge of all other routers, their links, and associated costs (metrics) within a network, stored in a shared topology database. 4

Link-State Advertisement (LSA) A fundamental data structure in OSPF used to exchange routing and topology information between routers. LSAs describe the state of router links and are distributed throughout an OSPF area to build a consistent Link State Database (LSDB). Each LSA type serves a specific purpose, representing different aspects of the network state, and is propagated to defined scopes such as individual routers, areas, or the entire autonomous system. 4, 5

mission control operator An employee of our industrial partner who works in a support role. They are typically the first to encounter a problem but may not necessarily be a specialist in the field. 36, 48

Multi-Access Network A network segment where multiple devices can communicate directly with each other over a shared medium (LAN). 4

Network LSA (Type 2 LSA) A network-LSA is generated for every transit broadcast or NBMA network by the Designated Router (DR). The network-LSA describes all the routers that are attached to the network. Visit [RFC2328, Chapter Network-LSAs](#) for details. 12

NSSA External LSA (Type 7 LSA) NSSA-external-LSAs are used in Not-So-Stubby Areas (NSSAs) to describe routes to external destinations. These LSAs are similar to Type 5 LSAs but are confined to the NSSA area and are converted to Type 5 LSAs by ABRs. Reference: [RFC3101](#). 13, 18, 45

Open Shortest Path First (OSPF) A dynamic routing protocol based on link-state technology, utilizing Dijkstra's algorithm to determine the shortest path. OSPF is widely used within autonomous systems for efficient IP routing. For a comprehensive overview and deeper insights, visit [OSPF on Wikipedia](#). ii, 3

OSPF Neighbors OSPF routers form neighbor relationships with adjacent routers on the same network segment. Neighbors progress through several states (Down, Init, 2-Way, Full) before achieving full adjacency. 13

Overadvertised Route A route that is being announced (advertised) to other devices in the network but should not be. This can cause confusion or unnecessary traffic in the network, as devices may try to use a route that is not supposed to exist. 7, 19, 29, 45

Prometheus An open-source monitoring and alerting toolkit designed for collecting, storing, and querying time-series data. It features a powerful query language (PromQL) and integrates well with cloud-native environments. 3, 11, 12, 19

Prometheus Node Exporter . 7, 27

Route List The complete set of routes known to an OSPF router, including intra-area, inter-area, and external routes, along with their metrics and next-hop information. 13

Router LSA (Type 1 LSA) A router originates a router-LSA for each area that it belongs to. Such an LSA describes the collected states of the router's links to the area. The LSA is flooded throughout the particular area, and no further. For a comprehensive overview and deeper insights, visit [RFC2328, Chapter Router-LSAs](#). 5, 6, 12, 18, 39, 40, 45, 46, 57, 60

Routing Information Base (RIB) A data structure used by routers to store all received and known routing information, including multiple possible routes to the same destination. These routes may contain duplicates and are evaluated based on predefined metrics to determine the most optimal path, which is then installed to the Forwarding Information Base (FIB) for actual packet forwarding. 4, 5, 14, 16

Stub Network An OSPF network type where no router adjacencies are formed, meaning transit traffic is not allowed. It can be seen as a client network. 5

Summary LSA (Type 3 LSA) Summary-LSAs are generated by Area Border Routers (ABRs) to describe inter-area routes. These LSAs are flooded throughout the backbone area and into connected non-backbone areas. For details, see [RFC2328, Chapter Summary-LSAs](#). 13

Transit Network An OSPF network type that connects two or more routers capable of forming adjacencies, typically represented by a Network LSA (Type 2 LSA). Visit [RFC2328](#) for details. 46

Unadvertised Route A route that is expected to be announced (advertised) to other devices in the network but is missing. This can cause connectivity issues because other devices are unaware of the route and cannot send data to the intended destination. 7, 19, 29, 45

Unix Domain Socket A communication endpoint that enables efficient inter-process communication (IPC) on the same host system, often used for exchanging data between client and server applications without relying on network protocols. 8, 28, 66

vtysh VTYSH is a shell for FRR daemons. It amalgamates all the CLI commands defined in each of the daemons and presents them to the user in a single shell, which saves the user from having to telnet to each of the daemons and use their individual shells. [More information here](#). ii, 8, 14, 15, 21, 47

Wrongly Advertised Route A route that is announced (advertised) with incorrect information, such as the wrong destination or subnet mask. This can lead to data being sent to the wrong location or being unable to reach its intended destination. 45

Acronyms

API Application Programming Interface. 7

BDR Backup Designated Router. 4

BGP Border Gateway Protocol (see Border Gateway Protocol (BGP)). 4, 9

DR Designated Router. 4, 46

FRR Free Range Routing. ii, iii, 4, 5, 7, 10, 40, 44

FRR-MAD Free Range Routing - Monitoring and Anomaly Detection. 7, 9

FRs Functional Requirements. 3

LSA Link State Advertisement (see Link-State Advertisement (LSA)). 6

LSDB Link-State Database (see Link State Database (LSDB)). ii, 4–7, 15, 18, 37, 57

OSPF Open Shortest Path First (see Open Shortest Path First (OSPF)). ii, 4, 7–10, 39

RIB Routing Information Base (see Routing Information Base (RIB)). 5, 6, 21

SLA Service Level Agreements. 6

TUI Text-Based User Interface. 8, 13, 21, 22, 37

Part II

Technical Report

Chapter 1

Introduction

This chapter introduces the context and scope of this bachelor's thesis. It begins by outlining the overall structure of the thesis, followed by a discussion of key terms and techniques used throughout the document. These terms are explained only to the extent necessary for understanding the content. Subsequently, the section on aims and objectives provides a concise overview of the core issues addressed and the goals pursued. Altogether, this chapter offers a clear starting point for what the thesis aims to achieve.

1.1 General

This thesis is structured into two main segments: a Technical Report and Project Documentation.

1.1.1 Technical Report

The Technical Report is the first part of this thesis and is divided into three chapters. The first chapter is the Introduction and familiarizes the reader with the overall context of the project. The three sections present the structure of the thesis, elaborate on key terms and techniques used, and define the aims and objectives of the work.

The second chapter of the technical report, Results, examines the results in detail. It begins by distinguishing this work from existing solutions and emphasizes the uniqueness of the presented approach. In the following section the Achievements are listed. Many objectives have been achieved and they are detailed, to demonstrate the capabilities of the solution. Finally, the chapter concludes with an in-depth analysis of the Implementation.

The final chapter of the technical report, Conclusion, evaluates the project outcomes and assesses the predefined goals. It also outlines potential directions for future improvements.

1.1.2 Project Documentation

During the four-month development period of this thesis, numerous strategic and technical decisions were made. While these decisions are grounded in technical expertise, they are not addressed in the concise technical report, as they fall outside its focused scope.

The Project Documentation begins by outlining the Requirements of the industry partner. While the Project Achievements section presents the fulfilled requirements from a user-oriented perspective, showcasing what the system delivers, the Requirements chapter documents the original Functional Requirements (FRs) as they were defined during the elaboration phase in collaboration with the industry partner. The four chapters that follow provide a structured account of the project's strategic approach. It covers tool and framework selection, architectural decisions, design rationale, quality assurance practices, and documentation standards. The Testing chapter ensures comprehensive validation of the thesis deliverables.

The concluding chapters of the project documentation address project planning, strategic development, and time management tracking.

1.2 Terms & Techniques

In this section, key concepts are presented to provide the necessary context for this thesis.

1.2.1 FRR-MAD

Free Range Routing - Monitoring and Anomaly Detection (FRR-MAD) is the name of the solution developed in this bachelor's thesis and consists of two main components, described below.

frr-mad-analyzer is an open-source Open Shortest Path First (OSPF) state analysis and information export tool. The **frr-mad-analyzer** is the daemon of this solution and is the first out of two tools resulting from this project. Provided Free Range Routing is running and OSPF enabled, this tool operates stand-alone. It's designed to analyze the state of an OSPF router and export the data in a format that Prometheus can understand. It consists of multiple components that act together as one service. The components are respectively: **aggregator**, **analyzer**, **exporter** and **socket**.

→ In the context of this thesis, **backend** is synonymous to **frr-mad-analyzer**.

frr-mad-tui is an open-source text-based user interface that complements the daemon **frr-mad-analyzer**. While the backend is a stand-alone daemon, the **frr-mad-tui** differs in nature. It's an extension to **frr-mad-analyzer** and only functions when the analyzer is running. It provides a text-based user interface to monitor any OSPF related anomalies or the OSPF and FIB state.

→ In the context of this thesis, **frontend** is synonymous to **frr-mad-tui**.

1.2.2 Free Range Routing

Free Range Routing (FRR) is an open-source IP routing protocol suite designed to provide high performance and flexible routing solutions for modern network infrastructures. It supports a wide range of routing protocols, including OSPF, BGP, RIP, IS-IS, and more, making it suitable for both enterprise and service provider environments. FRR is built on a modular architecture, allowing individual routing daemons to operate independently while sharing routing information through a central Routing Information Base (RIB).

Originally derived from the Quagga project, FRR has evolved with a strong focus on performance, scalability, and modern protocol support. It is actively maintained by a community of developers and contributors, including major networking vendors and open-source advocates.

In the context of this thesis, FRR is particularly relevant as it is the routing software suite deployed by the industry partner. Their infrastructure relies on FRR to implement OSPF, BGP, and maybe other routing functionalities, making it a critical component for understanding and evaluating the current state of the system as well as for developing improvements.

1.2.3 Open Shortest Path First

Open Shortest Path First (OSPF) is regarded as a legacy routing protocol. Its foundational specification, detailed in [RFC2328](#), dates back to 1998. Despite its age, OSPF continues to play a significant role in contemporary networking solutions, as emphasized by the industry partner Open Systems. OSPF is a widely used Interior Gateway Protocol (IGP) based on the Link State Routing Algorithm, which dynamically distributes routing information within autonomous systems to efficiently determine optimal paths for data transmission. In such networks, OSPF routers maintain identical copies of a Link-State Database (LSDB), which is a detailed repository containing topology information about the entire network segment. To prevent excessive flooding of Link-State Advertisement (LSA) among multiple routers in Multi-Access Network environments, OSPF employs an election process to select a Designated Router (DR) and a Backup Designated Router (BDR). The DR is responsible for consolidating and distributing routing information to other routers, significantly reducing the amount of protocol-related network traffic. The BDR acts as a standby, ensuring rapid convergence and continuity of routing information exchange if the DR becomes unavailable.

1.3 Aims and Objectives

This section outlines the issues addressed in this thesis and the corresponding solutions.

1.3.1 Problem

The existing monitoring infrastructure for Free Range Routing (FRR), particularly in the context of dynamic routing protocols such as OSPF and BGP, reveals substantial limitations. Current tools, including the `frr_exporter`, lack the analytical depth required to ensure consistency between the static

configuration and the actual operational state of the network. Beyond insufficient analytical capabilities, there is also a notable absence of mechanisms to effectively monitor the behavior of individual routers within the network.

These shortcomings hinder the detection and diagnosis of common routing issues such as missing, unexpected, or incorrect route advertisements, as well as duplicated entries. Furthermore, existing monitoring solutions do not adequately explain discrepancies between routes advertised in Link-State Advertisement (LSA) or configured in FRR and those ultimately installed in the Routing Information Base (RIB). This diagnostic gap limits the ability to troubleshoot routing anomalies in a timely and effective manner.

Common Routing Issues

To illustrate the routing issues consider the following router configuration:

```
1 !
2 interface eth1
3     ip address 10.0.0.1/24
4     ip ospf passive
5     ip ospf area 0.0.0.0
6 exit
7 !
8 interface eth2
9     ip address 10.10.10.1/24
10    ip ospf passive
11    ip ospf area 0.0.0.0
12 exit
13 !
14 router ospf
15     ospf router-id 1.1.1.1
16 exit
17 !
18 ! INFO: no other interface is configured on this router
```

Listing 1.1: Example Router Interface Config

Important to understand, that the routing issues can occur between configuration and LSDB, or between LSDB and RIB. This means inconsistencies may arise either during the translation of static configuration into Link-State Advertisement (LSA) or during the installation of routes from the LSDB into the Routing Information Base (RIB).

In case of static configuration translation, it's safe to say, that exactly these two network IDs of eth1 (10.0.0.0/24) and eth2 (10.10.10.0/24) must be advertised as Stub Network in the Router LSA (Type 1 LSA).

Issue	Example LSDB of Router LSA (Type 1 LSA)
Unadvertised A route that is expected to be announced (advertised) to other devices in the network but is missing. here: 10.10.10.0/24 is missing	Link connected to: Stub Network (Link ID) Net: 10.0.0.0 (Link Data) Network Mask: 255.255.255.0
Overadvertised A route that is being announced (advertised) unexpectedly to other devices in the network but should not be. here: 10.10.10.0/14 is unexpected	Link connected to: Stub Network (Link ID) Net: 10.0.0.0 (Link Data) Network Mask: 255.255.255.0 Link connected to: Stub Network (Link ID) Net: 10.10.10.0 (Link Data) Network Mask: 255.255.255.0 Link connected to: Stub Network (Link ID) Net: 10.10.10.0 (Link Data) Network Mask: 255.252.0.0
Duplicated Advertised A route that is announced (advertised) multiple times with conflicting attributes, such as different costs or paths. here: 10.10.10.0/24 is duplicated	Link connected to: Stub Network (Link ID) Net: 10.0.0.0 (Link Data) Network Mask: 255.255.255.0 Link connected to: Stub Network (Link ID) Net: 10.10.10.0 (Link Data) Network Mask: 255.255.255.0 Link connected to: Stub Network (Link ID) Net: 10.10.10.0 (Link Data) Network Mask: 255.255.255.0

Table 1.1: Routing Issues between Static Config and LSDB

To understand routing issues between the LSDB and the RIB, it's important to note that the RIB is allowed to contain multiple routes to the same prefix. Only the Forwarding Information Base (FIB) enforces a single best route per destination, typically the shortest path based on administrative distance and metric [6].

Therefore, verifying routing correctness requires analyzing the complete LSDB (including LSAs originated by other routers) and ensuring that every advertised prefix is present in the RIB as contributed by OSPF.

Industry Partner's Initial Situation

The industry partner operates all routers using [FRR version 8.5.4](#). Due to the critical nature of their services and strict Service Level Agreements (SLA), performing software upgrades (which typically require router restarts) is not feasible without risking service disruptions. As a result, they are constrained to using this specific version of FRR, which limits access to newer features, optimizations, and potential bug fixes available in later releases.

Our industry partner presented a specific problem encountered in their FRR environments, which ultimately served as the motivation for this thesis. The issue emerged during configuration changes and led to the occurrence of Unadvertised Routes and Overadvertised Routes routes. Further technical details can be found in the appendix.

Core Issues	Problem
Limited router-level OSPF monitoring: Resulting in time-consuming troubleshooting Missing (Unadvertised) routes Unexpected (Overadvertised) routes Version lock-in: Industry partner constrained to FRR 8.5.4	

1.3.2 Solution

The challenges mentioned above highlight the need for an enhanced anomaly detection system capable of generating timely alerts, a solution to visualize the routing configuration of FRR, and a mechanism to persistently store routing metrics for further analysis and correlation. To address these needs, the novel tool Free Range Routing - Monitoring and Anomaly Detection (FRR-MAD) is introduced, providing a focused set of essential capabilities for monitoring, visualization, and anomaly detection in OSPF-based FRR environments.

Due to time constraints, it was not feasible to implement all optional features, including the BGP-related enhancements. As a result, the current version of FRR-MAD is specifically focused on OSPF anomaly detection and monitoring.

We're excited to share that our industry partner, **Open Systems**, is already using FRR-MAD in production. Our team will continue maintaining and improving the project beyond the scope of this thesis. Several ideas for future enhancements are already in the pipeline and are outlined in the Future Directions section.

Analyzer Service - frr-mad-analyzer

One of the two major achievements of this bachelor's thesis is the frr-mad-analyzer. As the name indicates, it functions as the analytical part of the solution. It comprises four distinct components, each responsible for a specific role in the data processing pipeline.

- The aggregator component retrieves FRR operational data (including running configuration, LSDB data, and routing tables) and serializes it into structured data objects for consumption by downstream processes.
- The analyzer component processes the aggregator's structured data to compute predicted LSDB states and performs comparative analysis against actual network LSDB data. Analysis outputs are encapsulated as data objects accessible to both socket and exporter component.
- The exporter component exposes metrics through a Prometheus Node Exporter-compatible endpoint, enabling standard Prometheus instances to scrape network analysis data via the Application Programming Interface (API).

- The socket component establishes a Unix Domain Socket interface, providing the fr-mad-tui with real-time access to query operational data from the fr-mad-analyzer process.

Figure 1.1 provides an overview of how the four components interact. A detailed explanation follows in the Architecture subsection of the implementation.

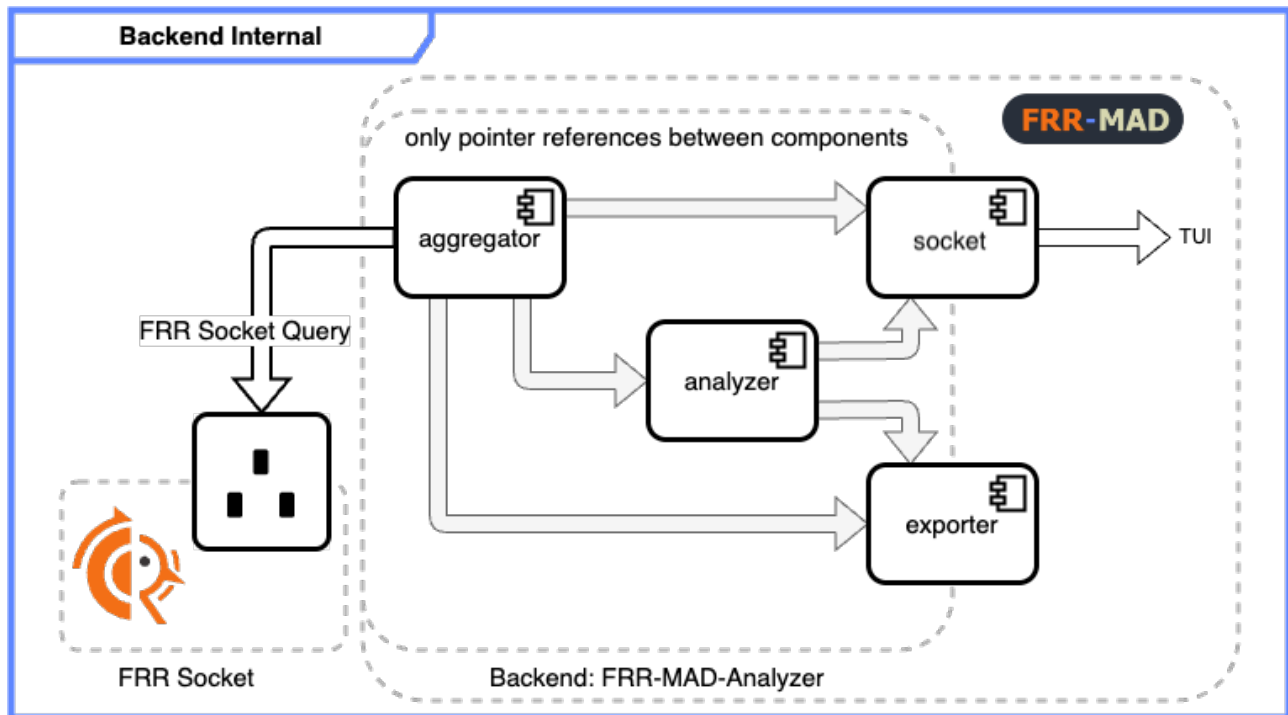


Figure 1.1: Design of fr-mad-analyzer Components

Monitoring Client - fr-mad-tui

The primary objective of this client is to provide a functional and intuitive Text-Based User Interface (TUI) that enables live monitoring of OSPF behavior and highlights any detected anomalies. It serves as a centralized interface, providing a comprehensive overview of the OSPF state on a given router. As a result, fr-mad-tui simplifies troubleshooting for network engineers and makes OSPF behavior more accessible to non-experts. Beyond its visual design (dashboard), the TUI provides practical functionality through features such as exporting parsed OSPF and FRR-related data to temporary files or directly to the system clipboard. Additionally, it supports direct execution of vtysh commands within the interface via the FRR-MAD TUI Write Mode.

The following figure demonstrate the FRR-MAD TUI, presenting the OSPF dashboard to illustrate the interface design.

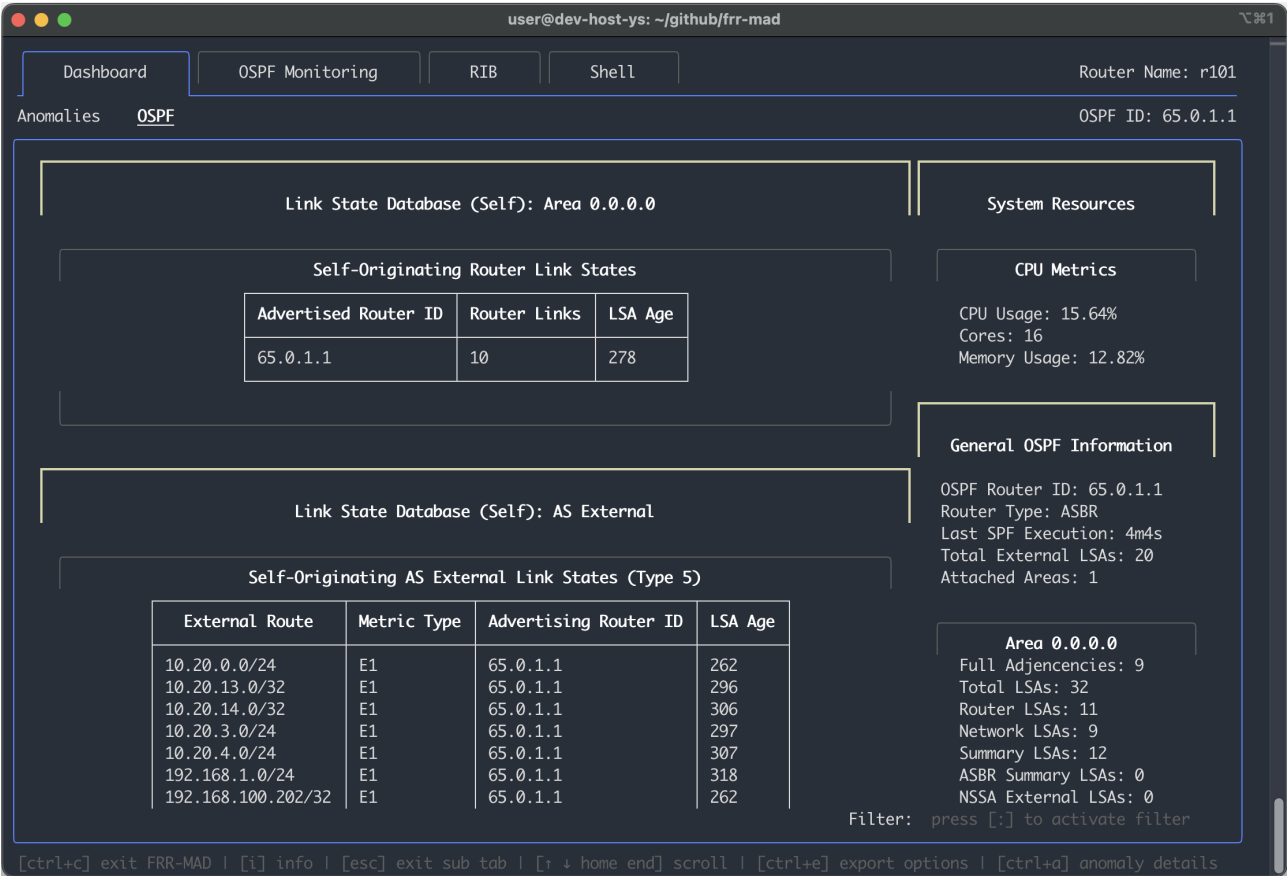


Figure 1.2: frr-mad-tui OSPF Dashboard

Development and Test Environment: Containerlab

To ensure seamless integration of all components, a significant part of this thesis focused on developing a comprehensive testing environment using containerlab.

The network topology includes multiple OSPF areas and BGP peers that interconnect distinct OSPF domains. The complexity of the topology was progressively expanded to accommodate the evolving capabilities of the thesis solution. The network concept is provided in the appendix, while the deployment files are available directly on [GitHub](#), along with detailed [documentation](#).

For both development and testing purposes, containerlab proved indispensable. During construction phases, the ability to rapidly test changes significantly enhanced development efficiency. Similarly, usability and acceptance testing were made feasible through this development environment.

Additionally, it serves as a ready-to-use test environment for potential users. The lab can be deployed with just a few commands, allowing anyone to quickly try out FRR-MAD in a realistic setup.

Core Artefacts	Solution
frr-mad-analyzer: Automated OSPF anomaly detection and metrics export	frr-mad-tui: Interactive TUI for live monitoring and troubleshooting
Develop and Test Environment: Containerlab-based OSPF lab for safe evaluation and demos	

Chapter 2

Results

This chapter examines the bachelor's thesis results in detail. The introduction briefly outlines the project boundaries. The two subsequent sections, Achievements and Implementation, detail the accomplished results and the final implementation respectively. The achievements section expands on the fulfilled requirements, while the implementation section describes the completed solution in comprehensive detail.

2.1 Project Boundaries

The Achievements and Implementation sections frequently reference Free Range Routing (FRR) and OSPF. It is important to clarify that no modifications to the implementation of FRR or OSPF were undertaken.

The implementations presented in this project represent original contributions developed by the three thesis authors. The FRR software suite and OSPF protocol specifications served as the foundational technologies for developing both backend and frontend system components.

This solution introduces a novel approach to monitoring and anomaly detection within OSPF network domains. At the time of the project's development, no existing tools were available that provided monitoring and analysis capabilities for OSPF behavior.

2.2 Project Achievements

This section elaborates on the solution's usage and the features achieved through its implementation. A separate FRR-MAD build and setup description is provided in the appendix.

2.2.1 Gather FRR Route Information

FRR-MAD implements a data collection process for routing information. The aggregator component receives structured JSON-formatted OSPF metrics from FRR at fixed intervals. Once parsed, this data becomes available to both the exporter and socket components. Metrics are exposed through a Prometheus-compatible endpoint for external polling via the exporter (discussed in subsection 2.2.2). Concurrently, `frr-mad-tui` accesses this data through the socket component. This collected routing data serves as the foundation for monitoring, anomaly detection, and subsequent analysis within the FRR-MAD system.

The following list provides a concise overview of achieved functions.

Unified OSPF Monitoring

- **Full LSDB Snapshots:** Complete Link State Database (LSDB) per area, including Router, Network, Summary, and External LSAs.
- **Neighbor States:** Live adjacency status (up/down), DR/BDR roles, and convergence metrics.
- **RIB/FIB Data:** Installed routes with protocol origins (OSPF/BGP/static), next hops, and metrics.

System Integration

- **FRR Configuration:** Static hostname, interface IPs, and OSPF areas from running configs.
- **Resource Metrics:** CPU/memory usage correlated with routing events.

Operational Benefits

- **No Manual Polling:** Metrics are gathered automatically at configurable intervals (default: 30s).
- **Structured Outputs:** Data is normalized for consistency across visualization (TUI) and export (Prometheus).
- **Anomaly Detection Ready:** Self-originating LSAs are pre-filtered for validation checks.

Supported Data Types

Category	Examples	Use Case
Topology	Router IDs, LSAs, neighbor states	Path validation
Routing Tables	RIB prefixes, next hops, admin distances	Route policy compliance
System Metrics	CPU load, memory usage	Capacity planning

Table 2.1: Supported FRR Data Types

Why This Matters

- **Troubleshooting:** Engineers can validate that current OSPF advertisements align with the router's configuration (e.g., passive interfaces are correctly advertised as stubs). However, the tool does not provide root cause analysis or historical correlation with specific OSPF events such as LSA floods or neighbor transitions.
- **Compliance:** Ensures that live OSPF state and advertised routes conform to intended design (e.g., redistribution rules, interface types).
- **Scalability:** Efficiently handles large-scale deployments, with normalized data structures and selective filtering (see Filter TUI content).

TL;DR

Gather FRR Route Information

FRR-MAD gathers and structures all relevant OSPF routing data, including LSDBs, neighbor states, and the routing table. This allows for high-resolution network monitoring, streamlined troubleshooting, and accurate anomaly detection across large-scale OSPF deployments.

2.2.2 Persistently store FRR Route Information

While not directly, FRR-MAD enables persistent storage of OSPF routing metrics via Prometheus. Export behavior is configurable through runtime flags or the application's configuration file. This modularity provides flexibility for users operating large-scale networks. This level of control helps manage Label Cardinality effectively, which is especially important in Prometheus environments with many routers.

Selective Metric Export (via startup flags)

Specific OSPF metrics can be enabled, by default is disabled, via command-line flags. In this example Router LSA (Type 1 LSA) and AS External LSA (Type 5 LSA) are exported:

```
frr-mad-analyzer start --ospf-router --ospf-external
```

Provided is a list of all options available.

- `--ospf-router` (Router LSA (Type 1 LSA))
- `--ospf-network` (Network LSA (Type 2 LSA))

- `--ospf-summary` (Summary LSA (Type 3 LSA))
- `--ospf-asbr-summary` (ASBR Summary LSA (Type 4 LSA))
- `--ospf-external` (AS External LSA (Type 5 LSA))
- `--ospf-nssa-external` (NSSA External LSA (Type 7 LSA))
- `--ospf-database` (Link State Database (LSDB))
- `--ospf-neighbors` (OSPF Neighbors)
- `--interface-list` (Interface List)
- `--route-list` (Route List)

Config File Based Metric Selection

Metrics can alternatively be configured using a YAML-based configuration file (the Cobra Configuration File used throughout FRR-MAD for all configurations).

2.2.3 Visualize FRR Route Information

This subsection first explains the structure of the TUI for gaining an effective overview of the system. It then describes the FRR routing information displayed on each page.

FRR-MAD-TUI Structure

All OSPF-related routing metrics gathered and exposed by FRR-MAD can be interactively viewed in the Text-Based User Interface (TUI) (`frr-mad-tui`). It presents FRR behavior on a router with a clear, engineer-focused design. The TUI is fully keyboard-driven, with the footer displaying real-time hints for actions and shortcuts.

Currently, the TUI is organized into four main pages, each serving a specific function. To reduce unnecessary scrolling (NFR8), pages include sub-pages or segmented views. (see Table 2.2)

Page	Purpose
Dashboard	Offers a high-level summary of OSPF metrics, system status, and routing data. If anomalies are detected, they are prominently displayed.
OSPF Monitor	Displays detailed information about OSPF neighbors, LSAs, and interface states. Serves as the core for OSPF-specific monitoring.
RIB	Presents a structured view of the current Routing Information Base (RIB) and Forwarding Information Base (FIB), including all installed routes and their attributes.
Shell	Enables direct interaction with FRR via FRR-MAD TUI Write Mode, including execution of vtysh commands and bash commands.

Table 2.2: Purpose of each frr-mad-tui Page

The Figure 2.1 shows the info page of the TUI, which can be accessed at any time by pressing the `i` key. This provides an overview of the main menu bar with the four previously described pages as well as all available keyboard shortcuts and user interactions.

In addition to navigation options, this page highlights important interface behaviors such as TUI modes (read/write) and message types (info, warning, error). It serves as a quick reference guide to help users efficiently operate the TUI.

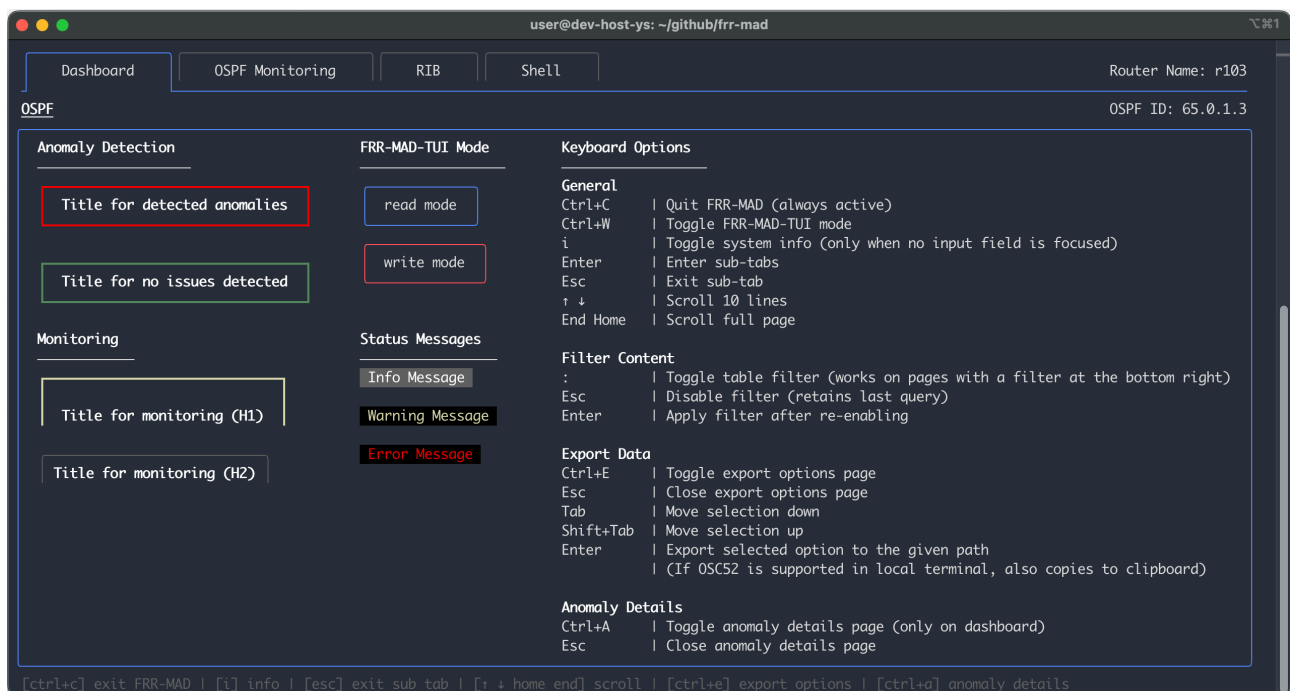


Figure 2.1: frr-mad-tui Info Page

Dashboard

The Dashboard currently contains two subpages. In the future, as more routing protocols are analyzed by FRR-MAD, each protocol will be provided with its own dedicated dashboard.

All dashboards are divided into two main panes:

- **Left Pane:** This pane is context-sensitive and updates based on the selected subpage.
 - If the anomaly dashboard is selected (default subpage of dashboards), the left pane displays anomaly-specific information.
 - The OSPF Dashboard is being used purely for monitoring, the left pane displays an overview using the vtysh command:

```
show ip ospf database self-originating
```
- **Right Pane:** This provides general system information, including:
 - **Resource utilization** (e.g., CPU, memory)
 - A summary of OSPF status retrieved with the command:

```
show ip ospf
```
 - Information about FRR-MAD like version and build-date

This layout ensures that both high-level system metrics and protocol-specific insights are readily available, supporting quick decision-making in both routine monitoring and anomaly scenarios.

OSPF Monitor

The OSPF Monitor page serves as the central hub for inspecting all relevant data required to monitor or troubleshoot OSPF behavior in detail. It is composed of several sub-pages, each focusing on a specific aspect of the OSPF protocol, allowing for targeted analysis and better operational insight.

- **LSDB Overview:** The first sub-page provides a full view of Link-State Database (LSDB), separated for each OSPF area.

```
show ip ospf database
```
- **Router LSA (Type 1):** This sub-page presents a detailed breakdown of Router LSAs. It categorizes and displays link information in separate tables based on link type: transit networks, stub networks, and point-to-point connections.

```
show ip ospf database router self-originating
```
- **Network LSA (Type 2):** Displays information about all Type 2 LSAs, which describe the routers connected to a shared broadcast or non-broadcast multi-access (NBMA) network.

```
show ip ospf database network self-originating
```
- **External LSAs (Type 5 and Type 7):** This sub-page provides detailed tables for external route advertisements, which are used to represent routes redistributed into OSPF from external sources such as BGP or static routes.

```
show ip ospf database external self-originating
```

```
show ip ospf database nssa-external self-originating
```

- **Neighbor Overview:** Presents comprehensive information about all OSPF neighbors.

```
show ip ospf neighbor
```

- **Running Configuration:** As a final sub-page, this view shows the live FRR running configuration directly within the TUI. It allows users to verify current OSPF-related settings without leaving the monitoring interface.

```
show running-config
```

RIB

The RIB page provides a detailed view of the router's Routing Information Base (RIB) and Forwarding Information Base (FIB). This page is essential for understanding which routes have been learned, and which paths are ultimately used for forwarding.

It presents the following outputs:

- All known routes (prefix and next hops), their sources (e.g., OSPF, BGP, static), and associated metrics as a clearly arranged table.

```
show ip route
```

- The number of routes in each routing table, which are accurately determined with the route summary command.

```
show ip route summary
```

Given that routing tables can contain a large number of entries, this page integrates the TUI's filtering mechanism (see Filter Feature), which is especially useful here. Users can quickly isolate specific prefixes, next hops, or protocol entries using the interactive filter interface (activated by the `:` key).

TL;DR

Visualize Frr Route Information

The frr-mad-tui provides a comprehensive and structured interface for inspecting OSPF routing data and system state. With dedicated pages for the dashboard, OSPF monitoring, and RIB/FIB inspection, the TUI significantly enhances the visibility and manageability of routing operations. Its filtering functionality is especially valuable in large-scale environments, enabling efficient navigation of high-volume routing data. These features collectively turn the TUI into a practical and powerful tool for monitoring, troubleshooting, and understanding FRR-based OSPF networks.

2.2.4 Export Parsed FRR Route Information

The previous subsection focused on the visualization of OSPF metrics and routing data within the frr-mad-tui. This part explains how users can export the underlying parsed and aggregated data collected by the frr-mad-analyzer.

Description

The frr-mad-tui offers a convenient mechanism for exporting any currently displayed or page-specific routing data. Users can open the export menu from any page or sub-page in the TUI. The export op-

tions are context-sensitive: each page exposes exportable data relevant to its content. In most cases, this reflects the data returned by backend calls used to populate the page. In the remaining cases, direct API calls to the ospfd.vty socket provide additional information.

Two export targets are available:

- **Temporary file:** The data is written to a temporary file at a path configurable via the FRR-MAD configuration file (Listing 2.1 illustrates where to modify this path).
- **Clipboard:** When supported by the terminal, data can be copied directly to the system clipboard using the OSC52 Protocol. This functionality is also available during remote SSH sessions, provided the client terminal supports OSC52.

```
1 default:
2   tempfiles: /tmp/frr-mad
3   exportpath: /tmp/frr-mad/exports # adjust this to specify tui export path
4   logpath: /var/log/frr-mad
```

Listing 2.1: Adjust Export Path in Configuration File

How to

When opening the export options with `Ctrl+E` at any point in the frr-mad-tui, users can navigate through the available export targets using `Tab` and `Shift+Tab`. Pressing `Enter` triggers the export, and the user is immediately notified of the export target location. Exactly this is illustrated in Figure 2.2.

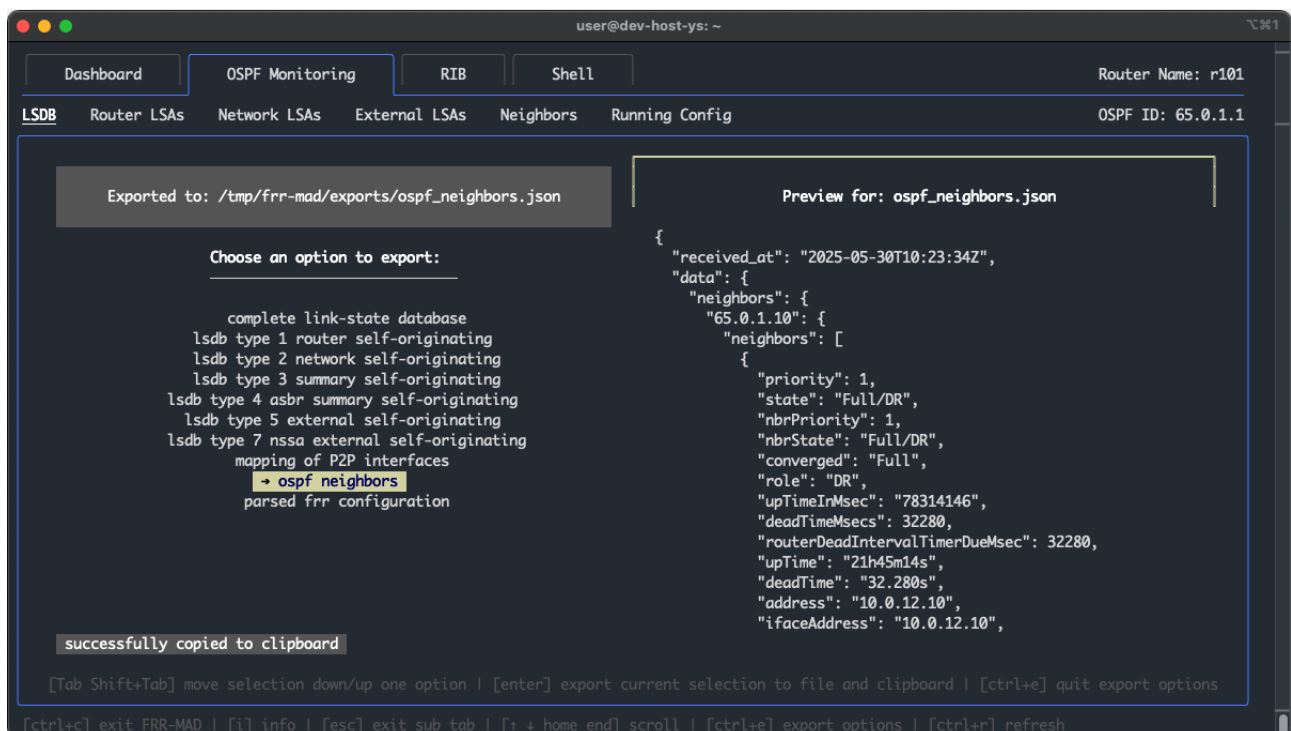


Figure 2.2: frr-mad-tui Export Page

TL;DR**Export Parsed FRR Route Information**

This export functionality enhances operational efficiency by enabling seamless transfer of routing snapshots, diagnostics, or metric outputs for further processing, sharing, or archiving. Any data parsed by the `frr-mad-analyzer` and used on the current page can be exported via the export menu, accessible with `Ctrl+E`. Data is exported to both the clipboard and a file located at a configurable path, as defined in the configuration file.

2.2.5 OSPF Anomaly Detection

One of the most important features is the detection of inconsistent behavior in FRR. For this purpose, `frr-mad-analyzer` compares two datasets to determine anomalies. The first dataset is the static FRR configuration file. Using the FRR configuration, predictions can be made for the LSDB, denoted as `shouldState`. The scope of these predictions is limited to self-originating routes only. Similarly, an `isState` is available from the router LSDB.

State Comparison

Table 2.3 presents a comparative analysis of the source metrics, designated as `shouldState` and `isState`, employed for anomalous behavior detection and analysis.

<code>shouldState</code>	<code>isState</code>	Description
FRR conf	LSDB Router	Provides the difference between the predicted Router LSA (Type 1 LSA) and the actual self-originating generated Type 1 LSDB. Contained within the same area.
FRR conf	LSDB External	Provides the difference between the predicted AS External LSA (Type 5 LSA) and the actual self-originating generated Type 5 LSDB. Exists in all areas except stub, totally stubby areas, and NSSAs, totally NSSAs.
FRR conf	LSDB NSSA-External	Provides the difference between the predicted NSSA External LSA (Type 7 LSA) and the actual self-originating generated Type 7 LSDB. Important for NSSA areas.
LSDB	FIB	Provides the difference between the populated LSDB and the actual FIB.

Table 2.3: Comparison of `isState` and `shouldState`

The concept is straightforward: compare the LSDB entries of Type 1, 5 and 7 against the configuration to identify discrepancies. The anomaly detection attempts to predict what should be added to the LSDB and then compares this prediction with the actual state.

While this approach is feasible, certain limitations must be acknowledged. The limitations will be explained in subsection 2.3.4

2.2.6 Export OSPF Anomalies

FRR-MAD exports all detected routing anomalies to Prometheus, providing immediate visibility into network issues without configuration overhead.

Anomaly Types Detected

As outlined in Table 2.3 there exists four different comparisons. These are further broken down into two major types of anomalies; Overadvertised Route and Unadvertised Route.

Category	Example Scenario	Severity
Over-Advertised	Static route announced in NSSA	Critical
Un-Advertised	Missing stub route	Critical

Table 2.4: Examples of Route Advertisement Issues

Operational Benefits

- **Live Alerts:** Includes prebuilt Prometheus alert rules
- **Forensic Data:** Export includes:
 - Source (Router/External/NSSA)
 - Link-State ID
 - Affected Interface
 - Protocol options

Key Differences to FRR Route Information

Table 2.5 distinguishes between configurable system features and those with preset configurations. Optional data points can thus be selectively hidden, such as detailed route metrics, while anomalies are always exported.

Feature	Route Metrics (2.2.3)	Anomalies (2.2.7)
Configurable	Yes (flags/config file)	No (always on)
Storage Use	Variable (based on selected metrics)	Fixed
Alerting	Manual alert rule setup	Preconfigured alert rules

Table 2.5: Comparison of Route Metrics and Anomalies Features

Note: Both data points are available at the same Prometheus endpoint :9091/metrics.

2.2.7 Visualize Detected Anomalies

Figure 2.3 illustrates how an anomaly is displayed within the dashboard. All previously described anomaly types are presented in this format. For additional details on a specific anomaly, users can press `Ctrl+A` within the dashboard to open a dedicated page that provides further descriptions of each anomaly type.

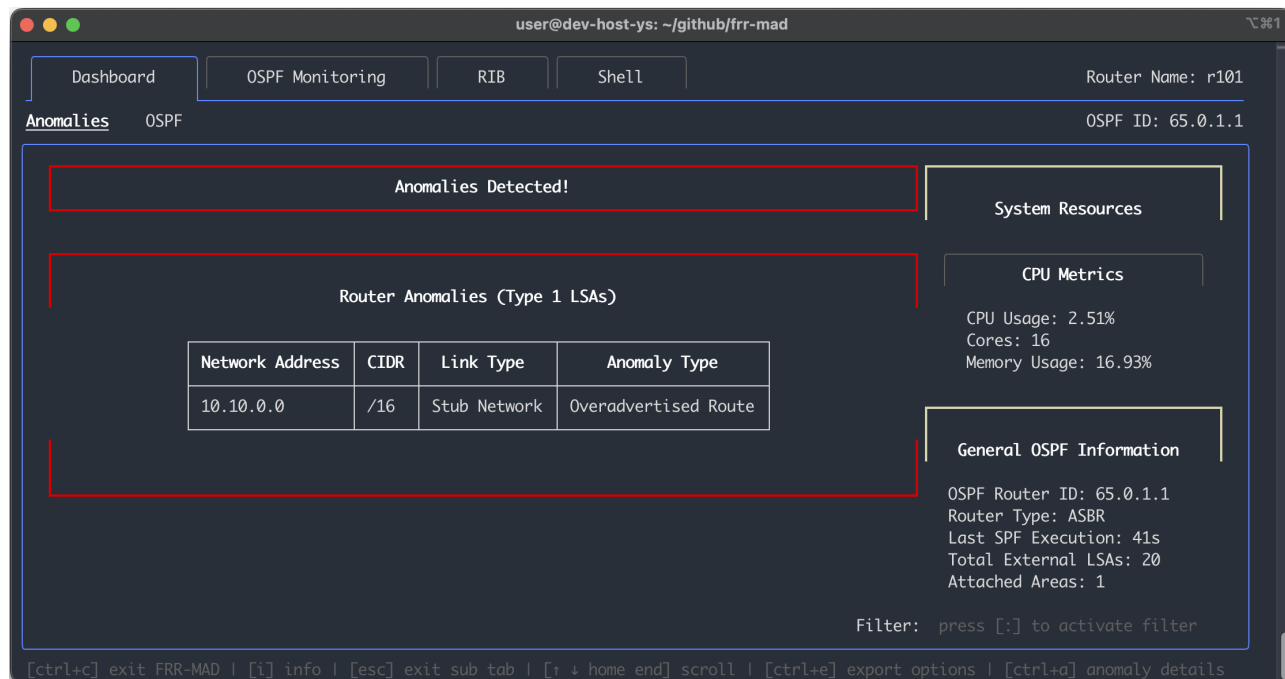


Figure 2.3: frr-mad-tui Anomaly Dashboard

2.2.8 Other TUI Features

The frr-mad-tui includes additional features that enhance the usability of both anomaly detection and OSPF monitoring.

Filter TUI Content

In large-scale network environments, such as those operated by service providers, the LSDB can grow significantly. Since OSPF does not define a strict upper limit for LSDB size, the number of entries is constrained only by the router's hardware resources. Practical deployments may contain anywhere from 1,000 to over 10,000 OSPF routes. Many vendors offer direct commands to limit the number of prefixes exported to OSPF. While FRR does not support this explicitly, similar control can be achieved using other methods[19].

To navigate and analyze such volumes efficiently, frr-mad-tui includes an interactive filtering function. This feature is essential for locating specific routes, LSAs, or configuration entries quickly. It greatly enhances usability and is particularly critical for troubleshooting and validating routing behavior in large and dynamic topologies.

As shown in Figure 2.4, the router maintains a Routing Information Base (RIB) containing 34 entries. The list is reduced to only two matching entries by applying a filter, which searches for the string `/23` marked in blue.

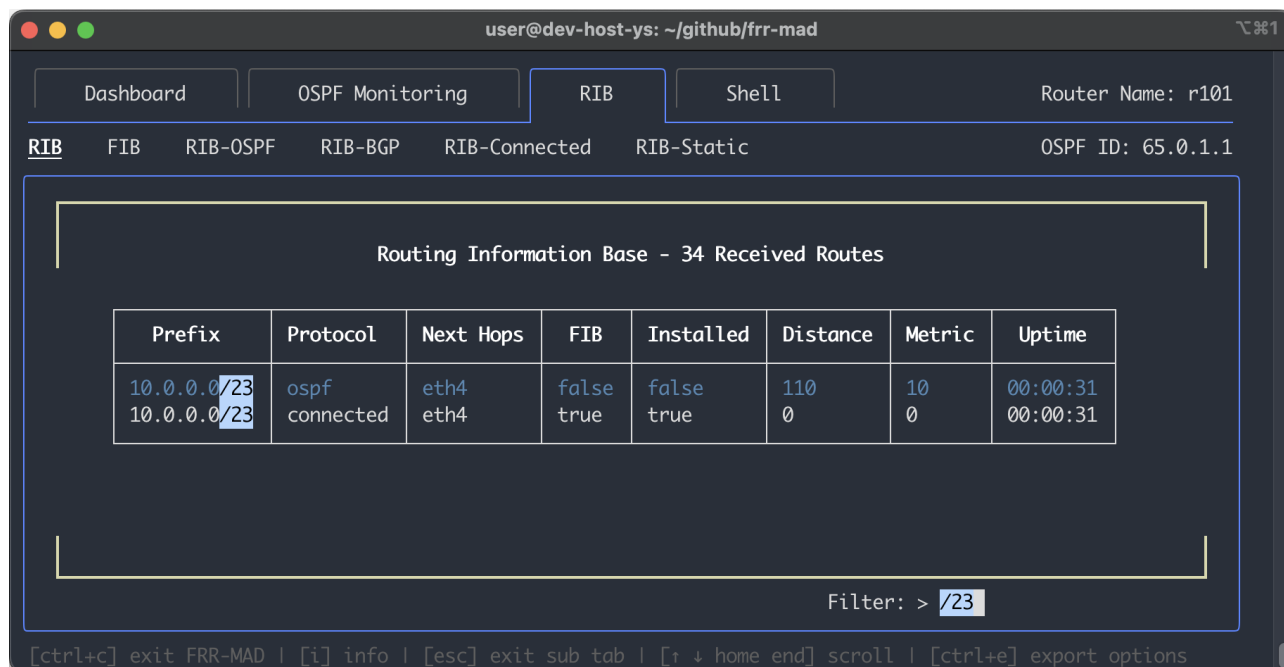


Figure 2.4: frr-mad-tui Filter Function

To activate the filter, press `:`. It can be deactivated either by pressing `:` again or `ESC`. If `ESC` is used, the current filter is retained and can be re-applied on any other page by pressing `:` again. In contrast, deactivating with `:` clears the filter immediately.

Custom Shell in TUI

The `frr-mad-tui` includes a custom shell interface to conduct troubleshooting workflows directly from the TUI. It consists of two subpages: a simplified shell and a `vttysh` interface. The simplified shell allows users to run basic bash commands such as `ip a` or `pwd`, while excluding commands that involve special characters (e.g., pipes, redirection). The second subpage provides full access to `vttysh`, enabling direct interaction with the FRR CLI. Figure 2.5 illustrates how the `vttysh` interface is used. Both shell interfaces require write mode to be enabled via `Ctrl+W`.

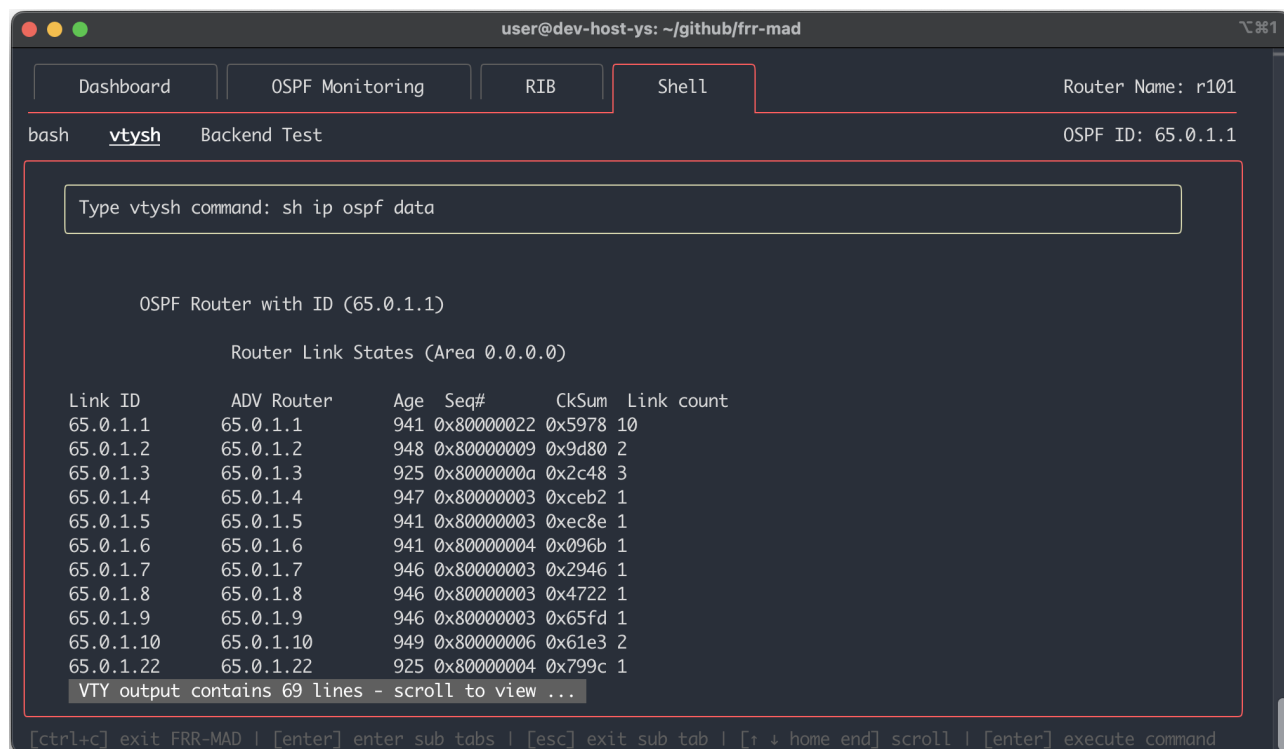


Figure 2.5: frr-mad-tui vttysh input

Status Bar TUI

The status bar is a persistent UI element located at the bottom of all TUI pages (except the running-config view). It provides system feedback and status updates.

Its two primary functions are:

- **System Alerts:** Color-coded messages indicate system state, including informational updates (e.g., successful export), warnings (e.g., high resource usage), and errors (e.g., failed API calls).

Info Message

Warning Message

Error Message

- **Contextual Guidance:** Dynamic hints are shown based on the current mode (e.g., read-only) or relevant actions.

The TUI automatically shortens long messages with ellipsis, based on the window size.

2.2.9 Augmented Development and Testing Environment

To conclude the achievements section, this project's development and testing environment solution is presented. To provide a system-independent test and development environment, containerlab has been utilized. Figure 2.6 illustrates the design of the development environment (see Appendix A.1 for full illustration). All routers and hosts are provided by Dockerfiles, and containerlab constructs the environment. A dedicated host was chosen to handle the build task, and through Docker mount processes, the build results are distributed across all routers.

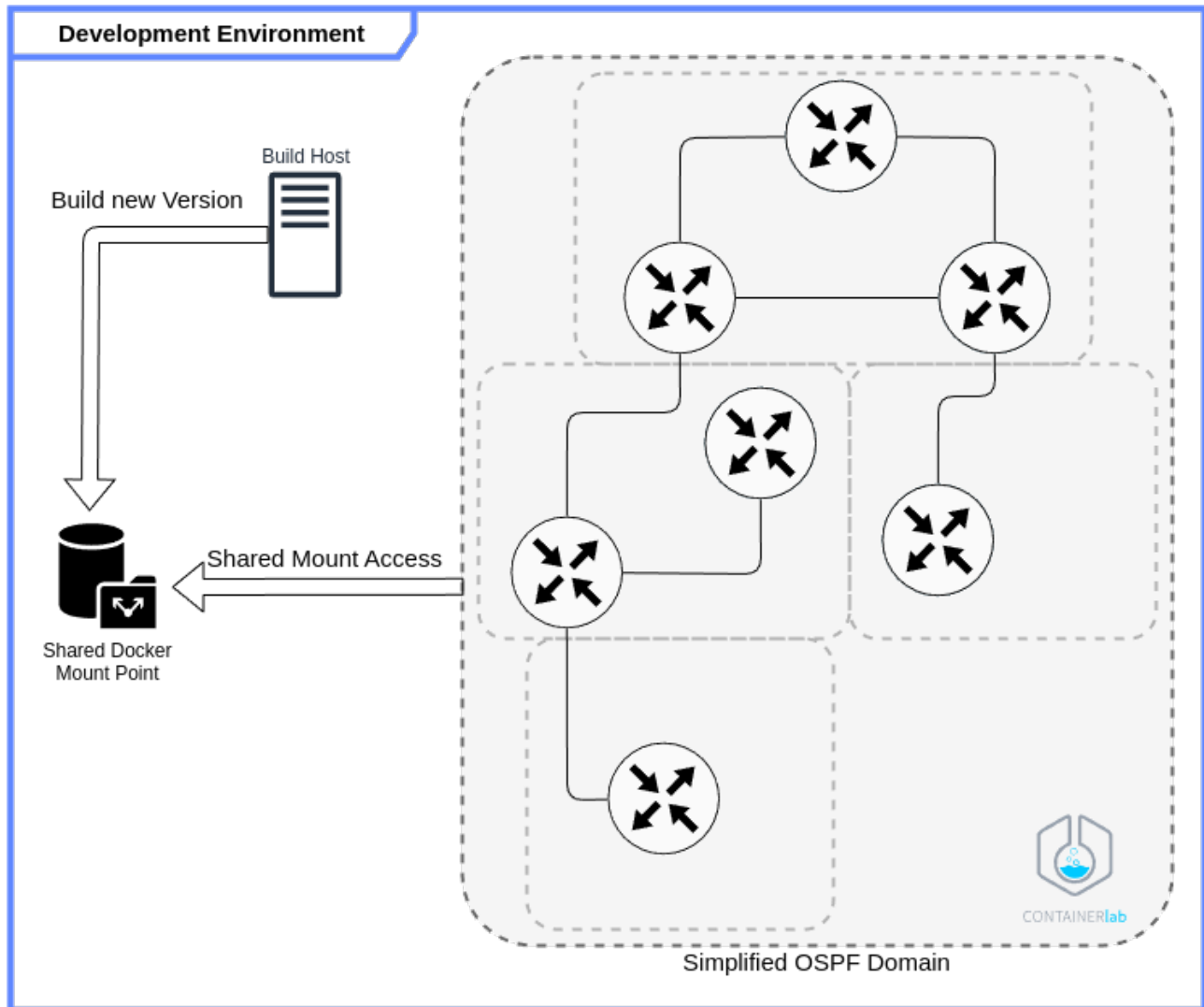


Figure 2.6: Development and Test Environment Overview

This design offers several advantages:

- System-independent development and building of files.
- Rapid replacement of build results across all containerlab devices.
- Dedicated build host reduces resource consumption compared to building on all containerlab hosts.
- Isolated build environment eliminates developer-specific build settings.
- Quick deployment and testing of the test environment.
- Enables quick, hands-on testing of FRR-MAD in a realistic setup with just a few commands.

2.3 Implementation

In the following section, the different aspects of implementing the Free Range Routing - Monitoring and Anomaly Detection (FRR-MAD) tool suit will be discussed in more detail. The overall approach

to reaching the solution will be outlined. Decisions on why certain technologies were chosen, are examined in the chapter Solution Strategy.

2.3.1 Architecture

The anomaly analysis is confined to the operation of a single device. As a result, the FRR-MAD software suite operates based on the FRR configuration and Link-State Database (LSDB) of an individual router. Figure 2.7 illustrates the interaction between the various components and services involved.

Although all illustrated entities are important, the implementation specifically focuses on the components `frr-mad-analyzer` and `frr-mad-tui`. The architectural abstraction offers a clear overview of the FRR-MAD tool suite's role within the overall system. While `frr-mad-analyzer` and `frr-mad-tui` form the core of the architecture, other referenced services and systems are included for context and to support a comprehensive understanding of the project's implementation.

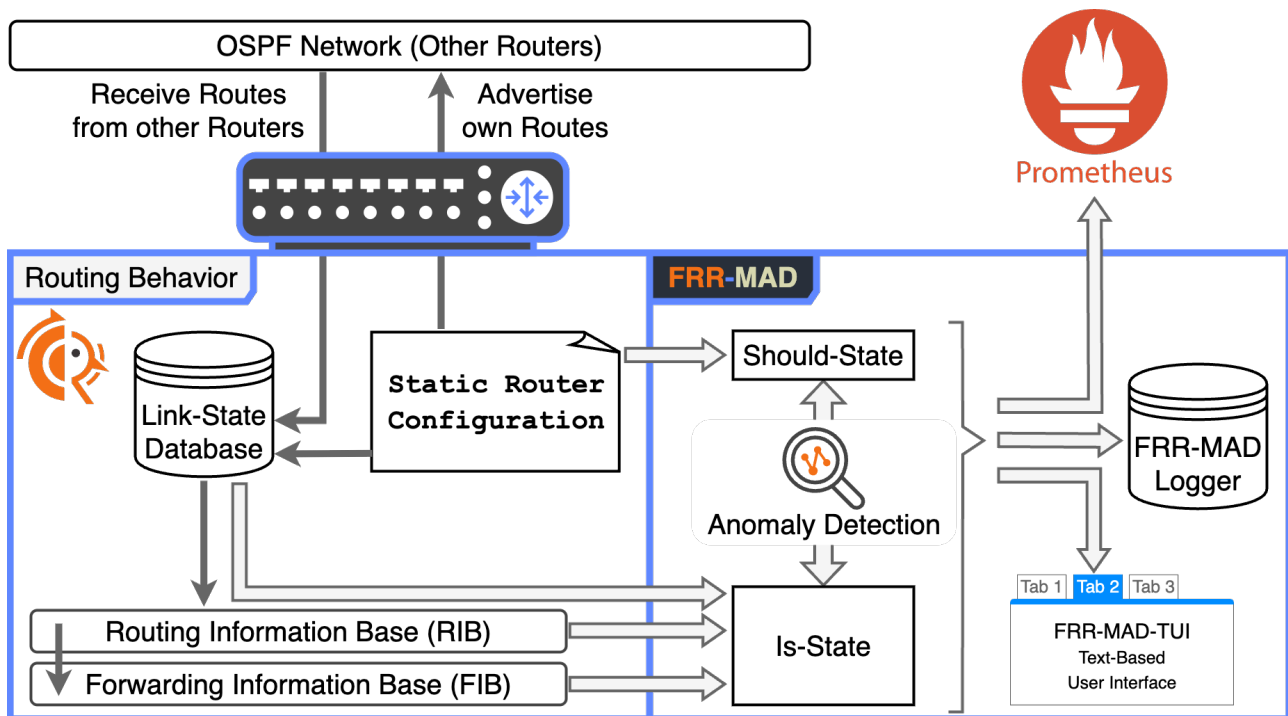


Figure 2.7: High Level Abstraction Architecture

Logger

The logging feature holds a distinct role within the FRR-MAD system. While it captures application-level behavior, it categorizes log messages into the following severity levels: debug, norm, info, warning, and error. In addition to general application behavior, inherent events within FRR-MAD, particularly those related to anomaly detection, are also recorded in the logs on the backend.

This logging mechanism enables more than just the use of export features from the `frr-mad-analyzer`. It allows users to directly inspect log files for any relevant state changes or anomalies. A similar ap-

proach is used for the `frr-mad-tui`, which shares the same logging foundation and stores all user interactions persistently in log files.

The log files are structured in JSON format, which facilitates efficient integration with systems that consume structured data. JSON is widely adopted due to its flexibility and readability. Each log entry includes additional metadata, most importantly, the service that generated the log and precise timestamps—ensuring traceability and consistency across the system.

Communication Channels

The communication scheme employs multiple implementation approaches, depending on the interaction layer.

FRR-MAD Suite ↔ FRR is the most critical communication channel. To enable effective anomaly detection, access to raw routing data is essential. FRR supports efficient querying via Unix sockets, which allows for low-latency communication using simple byte streams. This method minimizes overhead and enables rapid data retrieval.

While the use of byte-stream communication introduces certain limitations, most notably, the lack of built-in fault correction mechanisms—the performance benefits outweigh these drawbacks in the current context. The trade-off favors speed and simplicity, which are crucial for real-time analysis within the FRR-MAD system.

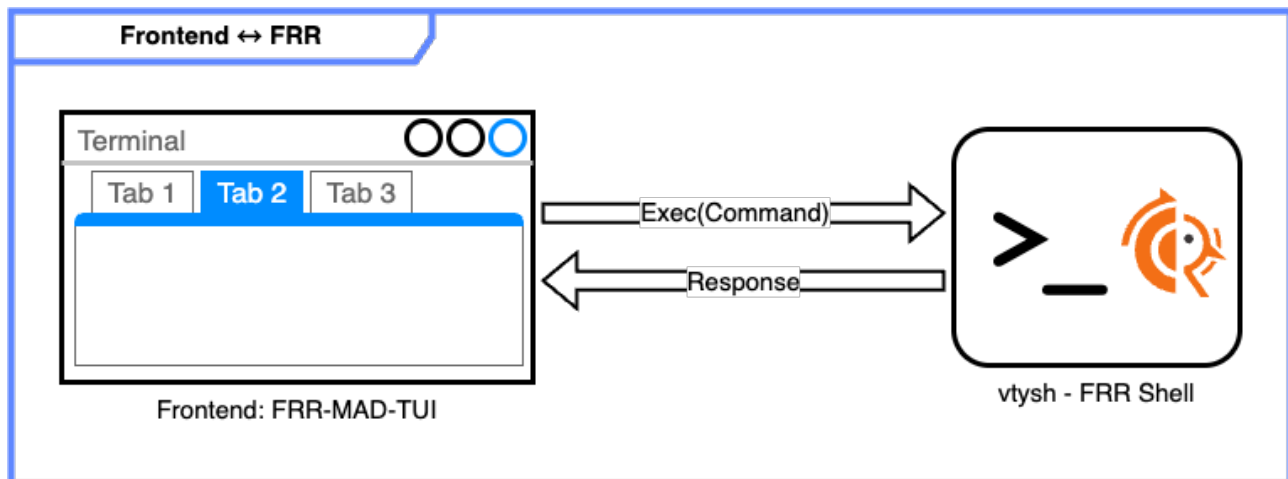


Figure 2.8: Frontend to FRR Communication

frr-mad-tui ↔ frr-mad-analyzer operates through decoupled communication. The frontend periodically queries the backend via Unix domain socket connections. The backend exposes a Unix socket endpoint for frontend state requests. This polling mechanism cannot reliably detect transient errors due to the absence of historical anomaly access. Two complementary systems address this limitation: the Logger and the exporter.

Both applications operate as standalone processes but require shared data object definitions. Data serialization is implemented using Protocol Buffers (protobuf) to ensure type consistency across process boundaries. Despite protobuf's native gRPC support, the implementation utilizes simple byte stream communication over the Unix socket, as shown in Figure 2.9. This design decision reflects the single-host, single-client communication pattern, which minimizes backend overhead. Given the local communication scope and fast information retrieval requirements, byte stream transmission provides sufficient performance without gRPC complexity.

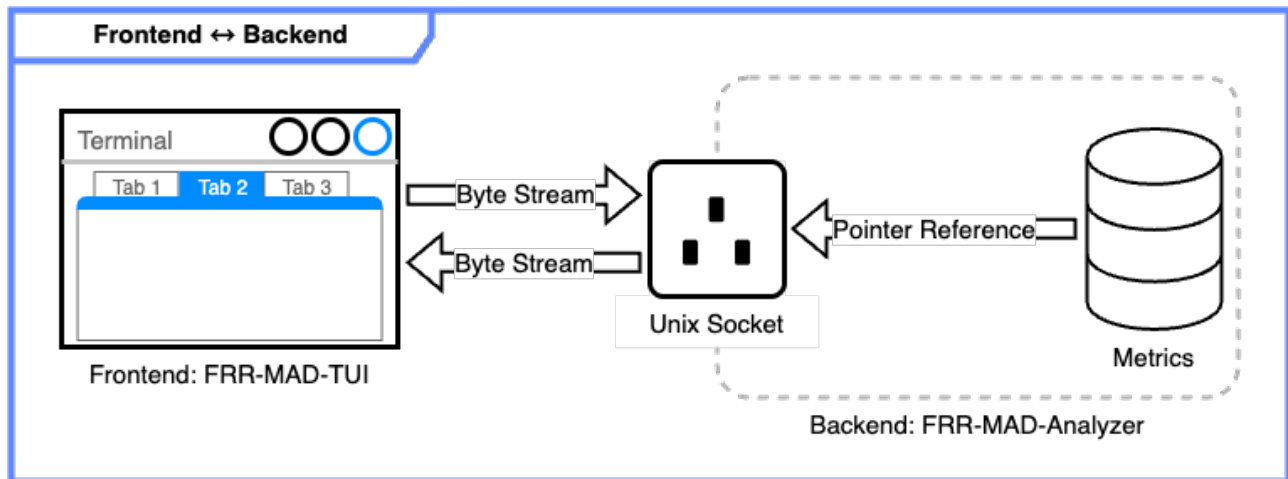


Figure 2.9: Frontend to Backend Communication

frr-mad-analyzer ↔ **internal communication** utilizes direct pointer references. Thanks to the use of pointer references, the application achieves a high degree of decoupling. The pointer references can be regarded as dependency injection. It provides the benefits that arbitrary data can be injected for testing purposes too. Once again protobuf is used for data serialization and handling.

The **frr-mad-analyzer** comprises four components: aggregator, analyzer, exporter, and communication handler. These components function as self-contained microservices with decoupled operational logic. Components expose pointers to protobuf-serialized data structures to maintain this architectural separation. An internal timer periodically refreshes data objects, ensuring all components access current metrics efficiently.

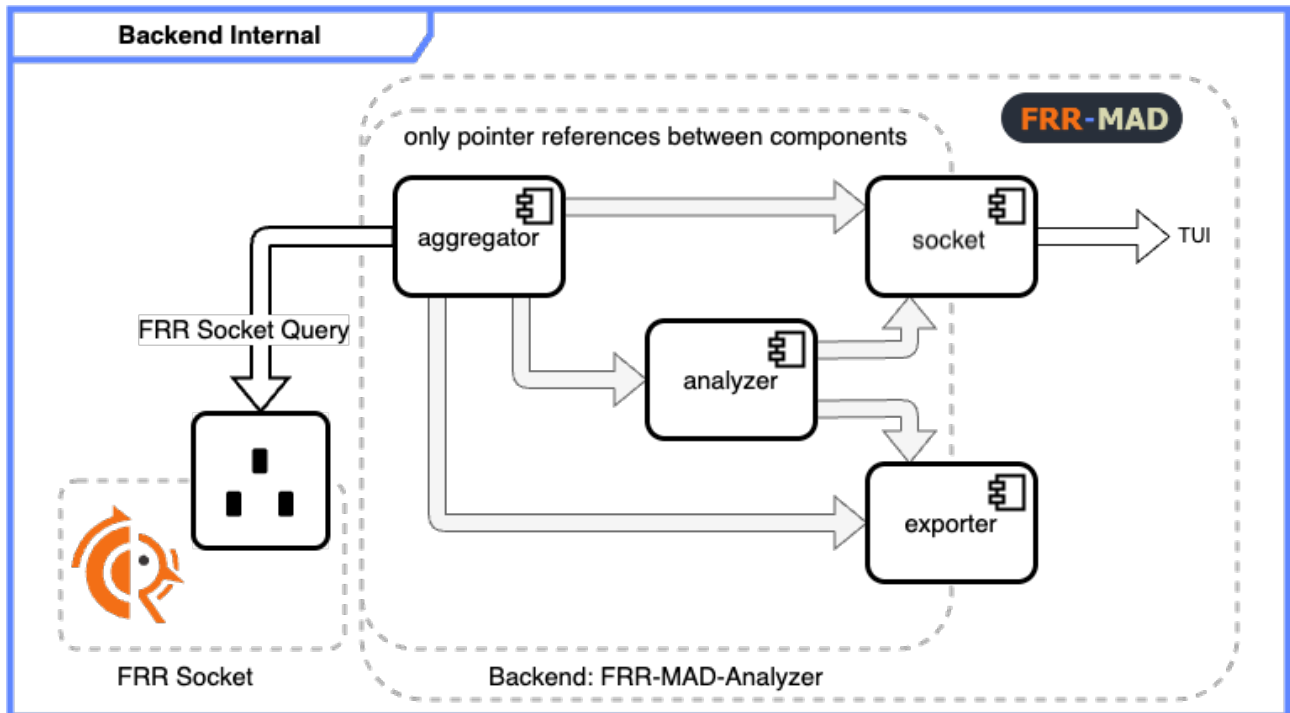


Figure 2.10: Backend internal communication

frr-mad-analyzer ↔ **Prometheus** implements the external metrics interface. This communication layer adheres to Prometheus Node Exporter endpoint specifications, as detailed in previous sections. As in internal communication explained, the metrics are made available with pointer references. The exporter exposes a copy of these metrics on a Prometheus compatible endpoint.

Figure 2.11 illustrates the communication structure. The Prometheus Node Exporter endpoint binds to all available network interfaces, enabling metric access via the configured port. The port number is configurable through the application configuration file.

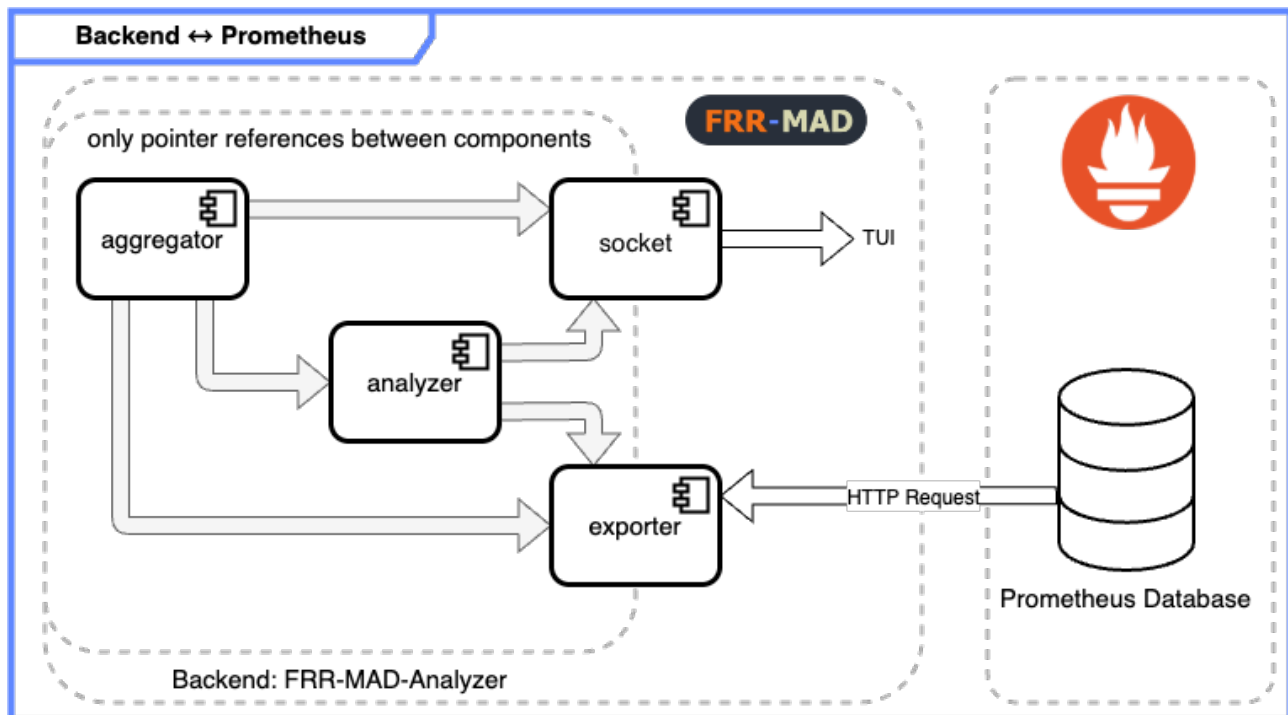


Figure 2.11: Backend - Prometheus Communication

2.3.2 Backend Service

The backend implements multiple communication paradigms. Direct synchronous communication with the frontend is established through a Unix Domain Socket interface. The exporter component facilitates the extraction and transmission of operational metrics from FRR services.

Prior to metric exportation, data collection and analysis must be performed. The aggregator component handles the retrieval and parsing of FRR operational data. The implementation follows a direct approach: information is obtained via FRR control sockets `zebra.vty` and `ospfd.vty`. Retrieved data undergoes parsing and serialization into structured data objects, which are subsequently made available to downstream components.

The following of this subsection examines the implementation of the analyzer, communications, and exporter components in detail. Refer to Section 2.2.5 to review the fundamentals of OSPF anomaly analysis.

Backend: Analyzer

The analysis process begins by generating a predicted Link State Database (LSDB) and comparing it against the actual LSDB. The prediction is derived by analyzing the FRR configuration and constructing the potential network states. This approach has an inherent limitation: only self-originating routes can be predicted using this method. Four distinct datasets are analyzed to identify potential anomalies. These datasets are calculated for LSA Types 1, 5, and 7, as well as the Forwarding Information Base (FIB).

Algorithm 1 provides an overview of the data preparation process. The analyzer receives parsed configuration data and different LSDB types as input. These two datasets have fundamentally different structures and must first be normalized to enable comparison. The most straightforward approach involves calculating the network addresses from the respective datasets and storing them in maps.

Algorithm 1: Convert OSPF Link States to Network Addresses Map

Input: *linkStateList* (collection of OSPF link states)

Output: *networkMap* (map of network addresses in CIDR notation with linkState information)

networkMap $\leftarrow$ [string]*linkState*;

foreach *linkState* $\in$ *linkStates* **do**

tmpLinkState := {

ip $\leftarrow$ EXTRACTIP(*linkState*);

mask $\leftarrow$ EXTRACTMASK(*linkState*);

network $\leftarrow$ CALCULATENETWORK(*ip*, *mask*);

cidr $\leftarrow$ CONVERTTOCIDR(*network*, *mask*);

networkSet $\leftarrow$ *networkSet* $\cup$ {*cidr*};

 } *networkMap*[*network*] = *tmpLinkState*;

end

return *networkMap*

The use of maps makes checking for the existence of link states straightforward. Algorithm 2 provides an overview of how the test is performed. If an entry from the *shouldState* is not present in the *isState*, an anomaly can be inferred and is classified as Unadvertised Route.

Algorithm 2: Detect Unadvertised Networks

Input: *shouldNetworkMap* (expected network addresses)

Input: *isNetworkMap* (actual network addresses)

Input: *unadvertisedList* (pointer to unadvertised networks list)

foreach *network* $\in$ *shouldNetworkMap* **do**

if *network* $\notin$ *isNetworkmap* **then**

 **unadvertisedList* $\leftarrow$ **unadvertisedList* $\cup$ {*network*};

end

end

Similar to the previous algorithm, Algorithm 3 abstracts the detection of Overadvertised Routes. The same logic applies, but in reverse. If a network from *isNetworkMap* is not present in *shouldNetworkMap*, it can be classified as overadvertised.

Algorithm 3: Detect Overadvertised Networks

Input: *shouldNetworkMap* (expected network addresses)
Input: *isNetworkMap* (actual network addresses)
Input: *overadvertisedList* (pointer to overadvertised networks list)

foreach *network* $\in$ *isNetworkMap* **do**
 if *network* $\notin$ *shouldNetworkMap* **then**
 **overadvertisedList* $\leftarrow$ **overadvertisedList* $\cup$ {*network*};
 end
end

The algorithms are executed sequentially. During the initialization of the different components, serialized data structures are created and shared between components. The information sharing process is therefore accomplished through pointer references, as previously mentioned. It is important to note that no two instances perform write operations on the same pointer reference. Only one component populates the pointer references while all other instances function as read-only consumers.

This testing procedure is generally sound, with one primary limitation: reduced accuracy. Different network types, such as stub or point-to-point networks, exhibit distinct behaviors.

While the algorithms presented above are high-level abstractions of the actual implementations, they accurately represent the general procedure. Ideally, the analyzer should infer the correct network type from the configuration, but this is not feasible with the current analyzer architecture.

Backend: Socket

As previously mentioned, the socket is implemented using a Unix socket. Byte streams transport protobuf-serialized data structures between components. Request messages are of type `Message` and contain the fields `Service` and `Command`. Algorithm 4 illustrates the general operation of the message handler. The socket receives a protobuf-serialized byte stream of type `Message` containing two data fields: `Service`, which determines the appropriate service handler for command routing, and `Command`, which specifies the requested information retrieval operation.

Algorithm 4: Unix Socket Message Handler

Input: $request \leftarrow Message\{\}$ (incoming bytestream; protobuf serialized data)
Output: $response \leftarrow Response\{\}$ (outgoing bytestream; protobuf serialized Response)

$message \leftarrow DESERIALIZEPROTOBUF(request, Message);$
 $service \leftarrow message.Service;$

switch $service$ **do**

case "ServiceA" **do**

$responseObj \leftarrow FORWARDTOSERVICEHANDLER("ServiceA", message);$

end

case "ServiceB" **do**

$responseObj \leftarrow FORWARDTOSERVICEHANDLER("ServiceB", message);$

end

case Default **do**

$responseObj \leftarrow CREATEERRORRESPONSE("InvalidService");$

end

end

$response \leftarrow SERIALIZEPROTOBUF(responseObj, Response);$

return $response$

Upon successful service identification, the corresponding service handler processes the Command. Algorithm 5 presents the generic implementation of the service handler. While each service handler follows a similar structure, their specific implementations do not allow for abstraction. However, this generic algorithm suffices for explaining the common service handler pattern. Once the message is properly parsed, requests are forwarded to their respective tasks. All subtasks consistently return Response objects as a design consideration to reduce complexity at the frontend-backend socket interface. The response object either contains a dataset corresponding to the Message request or returns `nil`. In addition, it reduces the complexity of testing.

Algorithm 5: Generic Service Handler

Input: *message* (deserialized Message object)
Output: *response* (outgoing byte stream; protobuf serialized Response)

command $\leftarrow$ *message.Command*;

switch *command* **do**

case "CommandA" **do**

result $\leftarrow$ EXECUTECOMMAND("CommandA", *message*);

responseObj $\leftarrow$ CREATESUCCESSRESPONSE(*result*);

end

case "CommandB" **do**

result $\leftarrow$ EXECUTECOMMAND("CommandB", *message*);

responseObj $\leftarrow$ CREATESUCCESSRESPONSE(*result*);

end

case Default **do**

responseObj $\leftarrow$ CREATEERRORRESPONSE("InvalidCommand");

end

end

response $\leftarrow$ SERIALIZEPROTOBUF(*responseObj*, Response);

return *response*

The communication system between these two components is designed to be synchronous. In the current iteration of FRR-MAD, there is no requirement for more complex capabilities. All information accessible to the frontend is also exported through a Prometheus-compatible endpoint.

Backend: Exporter

The exporter component serves as the metrics interface between FRR-MAD and external monitoring systems. It implements a Prometheus-compatible endpoint that exposes both operational metrics and anomaly detection results. The exporter follows a modular design with two primary subcomponents: the MetricExporter for FRR operational data and the AnomalyExporter for anomaly detection results.

Metrics Export

The MetricExporter takes the data from the pointer and exposes FRR operational metrics through a configurable HTTP endpoint. Key characteristics include:

- **Thread-safe Updates:** A read-write mutex protects metric updates during concurrent access
- **Comprehensive Coverage:** Supports exporting metrics for all LSA types including OSPF neighbor, Interface status and Routing table metrics

Algorithm 6 illustrates the metric update procedure. The exporter periodically refreshes all enabled metrics from the shared data structure while maintaining atomic operations through mutex protec-

tion. This algorithm uses a generic type parameter as a placeholder for any metric type that can be exported, with specific metrics outlined in Persistently Store FRR Route Information.

Algorithm 6: Metric Export Procedure

Input: *data* (shared FRR operational data)

Input: *config* (exporter configuration)

```

foreach metric category do
  | if category is enabled then
  | | switch category do
  | | | case generic do
  | | | | Update generic metrics;
  | | | end
  | | end
  | end
end
  
```

Anomaly Export

The AnomalyExporter provides detailed visibility into detected anomalies through three metric types:

- **Binary Flags:** Indicate presence/absence of anomaly types per detection source
- **Counters:** Track total anomalies detected per category
- **Detailed Metrics:** Provide labeled instances of specific anomalies

The exporter handles four anomaly sources:

- Router LSA anomalies (Type 1)
- External LSA anomalies (Type 5)
- NSSA LSA anomalies (Type 7)
- LSDB-to-RIB inconsistencies

Algorithm 7 shows the anomaly metric update process. The exporter first resets all metrics to zero before repopulating them with current anomaly data, ensuring metric staleness is prevented.

Algorithm 7: Anomaly Export Procedure**Input:** *anomalies* (shared anomaly detection results)

Reset all anomaly metrics to zero;

foreach *anomaly source* **do**

Update binary flags for detected anomaly types;

Update counters for anomaly counts;

foreach *detected anomaly* **do**

Create detailed metric with labels;

end**end****Implementation Details**

The exporter component implements several reliability features:

- **Retry Mechanism:** Failed metric updates automatically retry once before logging errors
- **Panic Recovery:** Catches and logs panics during metric updates
- **Atomic Operations:** Uses mutexes to prevent concurrent write conflicts

The HTTP server exposes metrics on a configurable port (default: 9091) with these endpoints:

- `/metrics` - Prometheus-formatted metrics
- `/` - Simple HTML status page

2.3.3 Frontend Client

The `frr-mad-tui` serves as the main interaction point for users. It is built using [Bubbletea](#), a robust and flexible Go framework for creating rich TUIs, and styled with [Lipgloss](#), which provides composable terminal styling utilities.

Frontend Structure

The source code for the TUI is organized for scalability, following a clean separation of concerns:

```

1 root/
2 +- src/
3     +- frontend/
4         +- cmd/
5             | +- tui/                # Entry point for the TUI application (main.go)
6         +- internal/
7             +- common/              # Shared types, utilities, and helper functions
8             +- pages/               # Page-based structure,
9             | +- examplePage/

```

```
10      |      +- model/      # Bubbletea model definition
11      |      +- update/   # Update logic and message handling
12      |      +- view/     # View rendering and data aggregation
13      +- services/        # Interfaces to the backend (e.g., socket
                           communication)
14      +- ui/              # Shared Lipgloss-based UI components and
                           layout styling
```

Listing 2.2: Frontend Structure

Design Philosophy

Each page within the TUI (e.g., Dashboard, OSPF Monitor) is implemented as an independent Bubbletea model, following a standardized pattern: model, update, and view. This structure promotes modularity and makes it easy to extend the TUI with new pages or features. Backend interactions are handled through the *services* layer, which abstracts the protobuf communication with the *frr-mad-analyzer*. Styling and layout components are centralized in the *ui* folder, which ensures consistency across all pages and enables quick design adjustments via Lipgloss.

Development Process

To become familiar with the framework and libraries, a minimal prototype was developed early in the project. While prior experience with CSS can be helpful, working with Lipgloss, and TUIs in general, presents its own unique challenges. One of the most time-consuming aspects was implementing responsive layout behavior. Unlike web development, Lipgloss does not offer built-in layout systems such as Flexbox or CSS Grid. Instead, all sizing must be handled manually in code. Functionality like `justify-content: space-between;` quickly loses its charm when you have to manually calculate the widths of container A and B, then insert a third invisible box as a spacer, just to make sure that A sticks to the left edge of the terminal and B hugs the right. Consequently, the *style.go* file contains extensive logic for calculating dynamic widths and heights, supporting various terminal segmentation (e.g., half, one-third, one-fourth).

Keywords	Frontend Client
Design Pattern: Model, View, Update	Bubbletea (Go Framework)
Modular Page Structure	Lipgloss Styling
Responsive Layout	Unix Socket Communication via Protobuf

2.3.4 Process and Challenges

This final subsection examines the key decisions and challenges encountered during the project development phase. The first section discusses the general workflow processes and design decisions made, along with subsequent adjustments implemented during development. The second section addresses the challenges and limitations encountered in anomaly detection implementation.

Workflow

This section outlines the workflow followed by a mission control operator when facing a routing problem. The corresponding flowchart serves as a foundational reference for later project phases, particularly for the development of the TUI-MVP.

To better understand the flowchart, several key aspects should be noted:

- Data export can be made at any point of the workflow.
- The "Anomaly Detection" process contains the anomalies according to FR3.

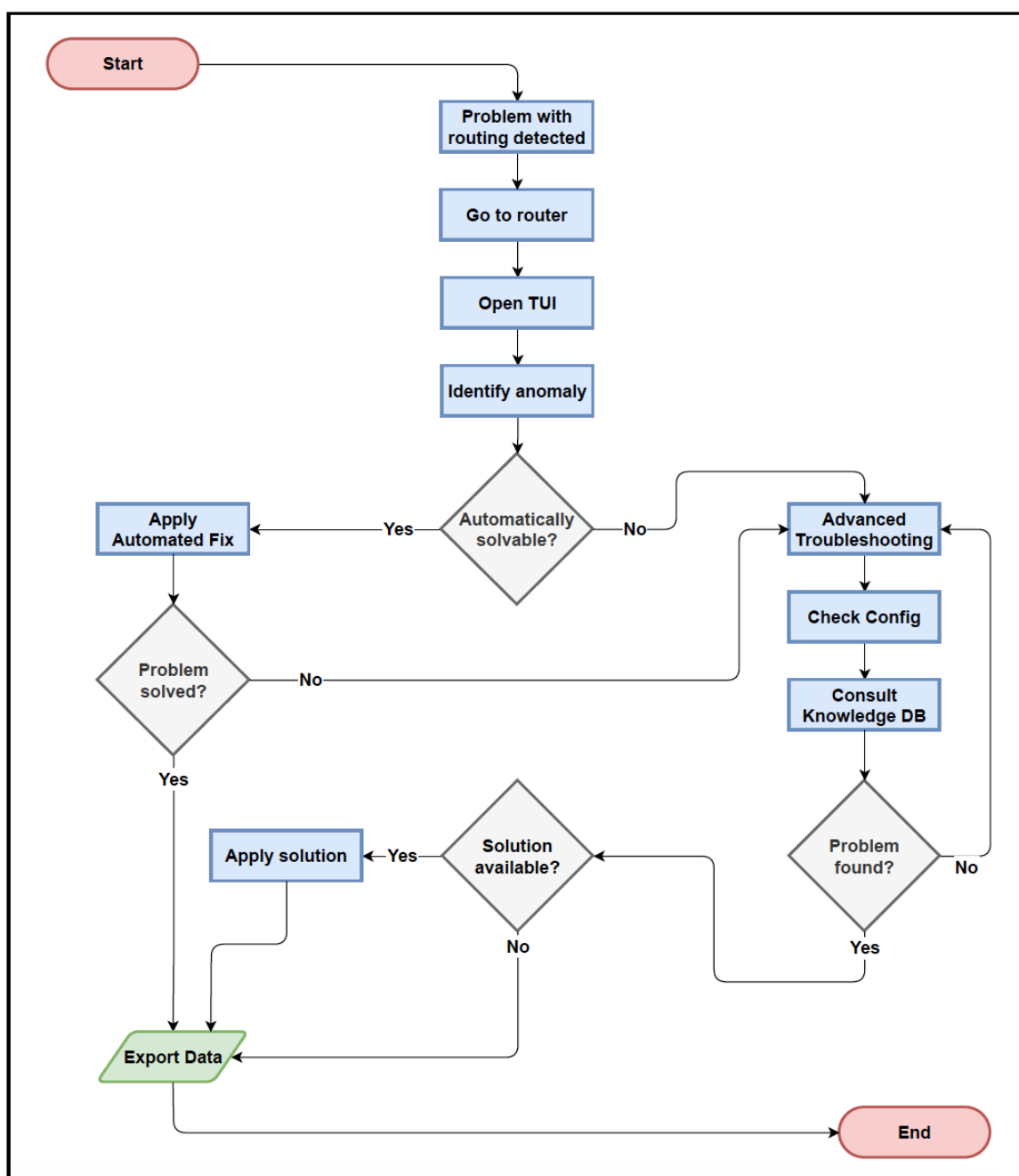


Figure 2.12: Workflow Flowchart

System Design Adjustments

The system design challenges encountered during development necessitated significant architectural modifications. Both Batfish and `frr_exporter` proved impractical for FRR-MAD development requirements.

Initial Design:

Figure 2.13 presents the initial system architecture following the project's elaboration phase. Initially, Batfish was evaluated as a potential engine for parsing and validating routing configurations of the complete OSPF network. Additionally, the `frr_exporter` was intended to handle all FRR-MAD-related analysis results export functionality.

By querying both `frr_exporter` and Batfish, it would obtain insights into network configuration and LSDB state. With this information the analysis process could be initiated as described in previous chapters. Finally, the `frr_exporter` was to be extended to export analysis results, while a Text-Based User Interface (TUI) would offer insights into the local router's state directly on the device.

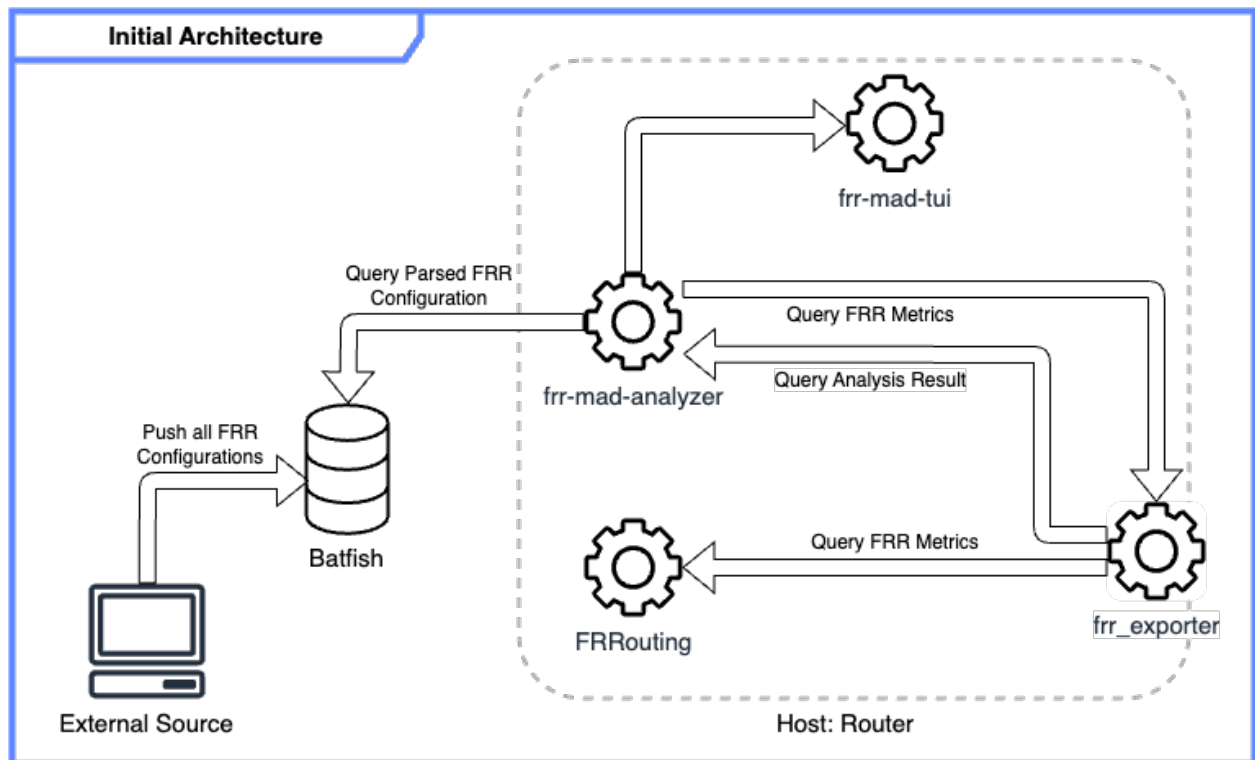


Figure 2.13: Elaboration Result: Initial Architecture

Adjusted Design:

Early in the development phase, it became evident that the above approach was insufficient. Both Batfish and `frr_exporter` proved to be suboptimal choices for implementation or adaptation.

Batfish required a dedicated host to perform network-wide FRR configuration validation within OSPF domains. This approach introduced practical limitations, as it would force users of the FRR-MAD toolset to deploy and maintain a centralized Batfish instance. This was neither desired by the industry partner nor aligned with our design goal to keep the solution self-contained and free of heavyweight dependencies.

The intended modifications to `frr_exporter` would violate its intended design principles. The `frr_exporter` is specifically designed to export FRR-based information without modification. The changes to `frr_exporter` by this project, would also see a strong coupling between the FRR-MAD toolset and `frr_exporter`. Based on these findings, the design shown in Figure 2.14 was chosen.

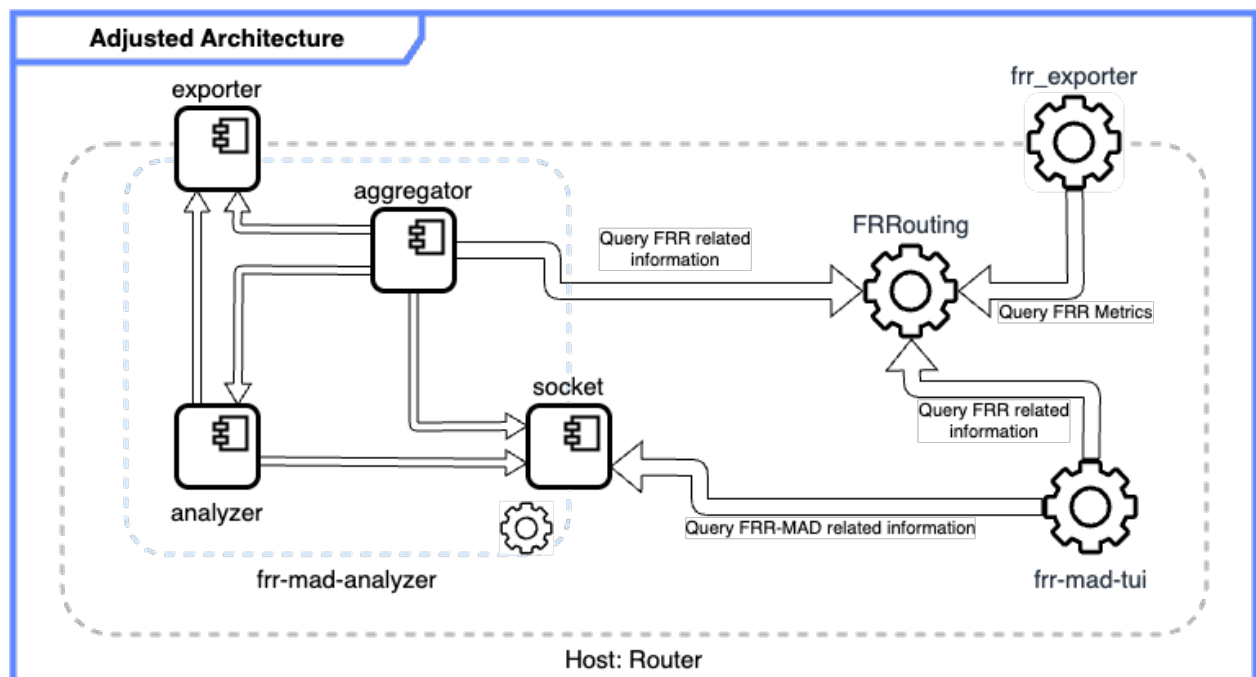


Figure 2.14: Development Phase: Adjusted Architecture

Anomaly Detection

The implementation of the analyzer logic required close collaboration with our industry partner, who provided essential domain knowledge. Since FRR-MAD is intended to run in this specific context, it was crucial to follow an agile development approach with rapid feedback loops. Despite extensive testing in our own lab environment (see Figure A.1), several issues only emerged in the industry partner's setup. These discrepancies highlighted two major challenges that arose from both a gap in our OSPF expertise and an initially limited understanding of the partner's infrastructure.

To clearly present the two problems, the explanation is structured into three parts: *Deep-Dive*, *Identified Problem*, and *Solution*.

Problem 1 - Predicted Link Type in Router LSA (Type 1 LSA)Deep Dive into Link Types:

A Router LSA (Type 1 LSA) advertises the state of a router's own interfaces. Within this LSA, each interface is assigned a link type, which depends on the interface configuration and the observed topology on the LAN. The general rule for determining the link type is shown in Table 2.7:

Interface Configuration	Situation on the LAN	Link Type in Router LSA (Type 1 LSA)
passive ¹	Only one Router (nothing else is possible)	stub network
no special network definitions	only one Router	stub network
no special network definitions	two or more routers connected to the same LAN	transit network

Table 2.7: Link Types based on Interface State

¹A passive interface is an interface that doesn't send or receive OSPF hello packets, but still participates in the routing process by learning routes from other OSPF routers on that interface.

Identified Problem:

The absence of a passive configuration on an interface does not necessarily indicate that it belongs to a transit network.

Solution:

We introduced a new network type called "unknown" for cases where an interface is expected to act as a transit network but cannot be confidently classified during prediction. If the shouldState marks a link as "unknown", and the actual isState turns out to be either a stub or transit network, this is treated as a valid match, no anomaly is raised in such cases.

Resulting Limitation:

Since the link type is yet another element in OSPF that can change dynamically at runtime, this solution introduces a limitation. It is technically possible for a non-passive interface to appear with a different link type than expected, without triggering an anomaly.

Problem 2 - Predicted Router LSA (Type 1 LSA) for P2P InterfacesDeep Dive Point-to-Point Networks:

The behavior is clearly defined in [RFC 2328, Section 12.4.1.2](#). The handling of interfaces in Router LSA (Type 1 LSA) generation depends on whether the interface is numbered or unnumbered:

Numbered ²	Unnumbered ³
Stub Network: If the neighboring router's IP address is known, the Link ID is set to that IP address. If the neighbor is not known, the subnet's IP address is used as the Link ID.	Point-to-Point: Only a single link is advertised with link type point-to-point. The Link ID is set to the interface index (ifIndex) of the local router's interface.
Point-to-Point: The Link ID is set to the Router ID of the neighboring router.	

Table 2.9: Difference of Router LSA (Type 1 LSA) from Numbered/Unnumbered Interfaces

²A numbered interface has an IP address and subnet assigned.

³An unnumbered interface has no IP address assigned; uses interface index as identifier.

Point-to-Point peerings in OSPF can be configured in several common ways:

- /32 Address: The neighbor's address must be manually specified, as no subnet is implied.
- /31 Network: The neighbor's IP address is automatically derived, as both usable addresses in the /31 subnet are assumed to be point-to-point peers. This is supported due to [RFC 3021](#) and is commonly used to conserve address space.
- /30 Network: The neighbor's address is automatically derived from the subnet. The network and broadcast addresses are reserved, so only two usable host addresses remain.

It is also important to note that FRR treats interfaces configured with a /32 address as unnumbered, which is documented in the [official FRR documentation](#).

Identified Problem:

The industry partner's configuration deviates from RFC-specified standards. Despite this deviation, their LSDB is populated in a manner that appears functionally correct. The underlying interface operates as a peer-to-peer interface, as confirmed by examining the applied configuration using the command `ip ospf interface <interfaceName>`. This behavior occurs because the underlying interface is inherently peer-to-peer, and the Zebra daemon correctly identifies this configuration.

Solution:

To address this issue, the anomaly analyzer was adjusted to treat the interface as numbered, even though the [official FRR documentation](#) indicates that such configurations should be interpreted as unnumbered. This adjustment accurately reflects the standard behavior observed with the industry partner's setup.

Resulting Limitation:

A significant issue remains unresolved. The peer-to-peer interface parsing logic was specifically tailored to accommodate the industry partner's configuration. Consequently, /32 point-to-point peering configurations will trigger false anomaly detection. This limitation will be addressed in future updates.

Anomaly Detection Limitations

Due to the decentralized nature of OSPF, the information available on a single host is limited to its local configuration scope. This constraint limits LSDB prediction capabilities to self-originated entries resulting from the router's own FRR configuration, such as interfaces and static routes. For example, self-originated AS External LSA (Type 5 LSA) entries redistributed from BGP cannot be predicted. Despite this limitation, it remains possible to reliably predict self-originating link states.

Once both the `isState` and `shouldState` have been parsed, the comparison process begins. At this point, the analysis encounters another challenge. Ideally, the anomaly detection should reliably determine whether a network functions as a stub or a transit network. In practice, however, this is not feasible with sufficient accuracy.

Chapter 3

Conclusion

This chapter evaluates the functional and non-functional requirements defined at the beginning of the project and concludes with an outlook on potential future developments of the FRR-MAD project beyond the scope of this thesis.

3.1 Outcome Analysis

This section presents a comprehensive evaluation of all defined Functional Requirements and Non-Functional Requirements from the project specification. Each requirement is individually assessed with regard to its implementation status, validation method, and critical discussion of outcomes and trade-offs. Optional features are also reviewed to determine their current level of completion and their potential for future development.

Legend for Evaluation Icons:



MVP and Finished



MVP and not Finished



Optional and Finished



Optional and not Finished

3.1.1 Functional Requirements Evaluation

✓ MVP | FR1-1: Export OSPF Routing Metrics

Detail	The backend (frr-mad-analyzer) supports exporting relevant OSPF metrics in JSON format, which are directly polled from the router. A Prometheus-compatible endpoint has been implemented to access these metrics externally, enabling persistent storage of these metrics. The configuration file allows users to selectively define which metrics are exported. Due to the improved architecture, these export settings do not interfere with any functionality in the TUI, even if no metrics are exported. Additionally, the backend (frr-mad-analyzer) is designed to be easily extensible, making it straightforward to add support for additional metrics if needed.
Status	Fully implemented
Validation	Unit Tests, Acceptance Test with stakeholder
Discussion	The initial implementation of this feature was static in nature. However, to accommodate the industry partner's request to reduce the number of Prometheus labels, flexibility was incorporated into this feature. Consequently, individual metrics can be enabled or disabled through either the configuration file or command-line options. This feature includes enhancements beyond the original specification.

✗ Optional | FR1-2: Export BGP Routing Metrics

Detail	This optional FR aimed to provide BGP metrics export functionality, similar to the OSPF implementation described in FR1-1 evaluation.
Status	Not implemented
Reason (if not completed)	Several challenges were encountered during the implementation of OSPF anomaly detection, as discussed in FR3 evaluation. Additionally, the scope of the MVP regarding both anomaly detection (FR3 evaluation) and information gathering (FR2 evaluation) was clearly underestimated.
Discussion	The decision to exclude BGP-related functionality was made in project week 12 and discussed with the stakeholder on May 8, 2025. This trade-off allowed the team to focus on completing and stabilizing OSPF-related features. Future work could include extending the system to support BGP metrics in a similar fashion to OSPF.

 MVP | **FR2:** Gather FRR Routing Information

Detail	The backend (frr-mad-analyzer) includes a dedicated module for aggregating OSPF routing information from Free Range Routing (FRR). Using a fixed polling interval of 5 seconds, the system continuously collects data such as installed and advertised OSPF routes, various LSA types (1, 2, 3, 4, 5 and 7), and FRR system configuration and status. The aggregated data is made available for real-time analysis in the analyzer component and for visualization in the front-end (frr-mad-tui).
Status	Fully implemented
Validation	Unit Tests, Acceptance test with stakeholder
Discussion	Due to the lack of open-source tools for parsing FRR routing data, we developed a custom parser using FRR's JSON command output. This solution integrated well and met project needs. However, structural changes in FRR version 10.3 JSON responses broke compatibility. As a result, FRR-MAD currently supports only the stable 8.5 release . A future enhancement would be to extend parser compatibility across multiple FRR versions.

✓ MVP | FR3: OSPF Route Anomaly Detection by TUI

Detail	The analyzer component implements OSPF route anomaly detection focusing on three key LSA types: Router LSA (Type 1 LSA), which check intra-area links and interfaces; AS External LSA (Type 5 LSA), which verify redistributed routes; and NSSA External LSA (Type 7 LSA), which validate NSSA-specific routes. Additionally, the system performs FIB-LSDB consistency checks to identify installed routes that don't match the LSDB. The detection covers unadvertised routes (missing expected routes) and overadvertised routes (unexpectedly announced routes).
Status	Partially implemented.
Validation	Unit tests for core detection logic and acceptance tests with stakeholder using real network scenarios
Reason (if not completed)	No examples were provided by the stakeholder to demonstrate how a Duplicated Route could occur. Additionally, no clear case was given for Wrongly Advertised Route that could not already be categorized as either Unadvertised Route or Overadvertised Route.
Discussion	The implementation provides comprehensive anomaly detection for the most critical LSA types affecting route propagation. The FIB-LSDB comparison adds valuable operational state validation. While Network (Type 2) and Summary (Type 3/4) LSAs aren't directly analyzed, their effects are captured through the router LSA and FIB checks. The modular design allows for future extension to additional LSA types if needed.
Comment	The system focuses on the LSA types that most directly impact route anomalies, providing targeted detection without unnecessary complexity

✓ MVP | FR4: Query and Display Routing Anomalies via TUI

Detail	The frontend (fr-mad-tui) visualizes all required OSPF anomalies as defined in FR4, including affected prefixes and expected vs. actual values. Additionally, it covers further anomalies such as LSDB-to-RIB and RIB-to-FIB mismatches. The dedicated OSPF Monitor and RIB pages were developed to support troubleshooting. Anomaly data is queried automatically at regular intervals or upon accessing the OSPF dashboard.
Status	Partially implemented.
Validation	Acceptance test with stakeholder
Reason (if not completed)	Same reason as in FR3 evaluation.

✕ Optional | FR5: BGP Route Anomaly Detection

Detail	-
Status	Not implemented
Comment	BGP-related functionality was entirely excluded from the project scope, as discussed in FR1-2 evaluation.

✓ Optional | FR6: Adding new tabs to TUI

Detail	The config file enables control over which pages of the frontend (frr-mad-tui) are active. For instance, the shell page can be disabled while retaining full functionality of the remaining pages. On startup, the TUI reads these settings and renders only the activated pages.
Status	Fully implemented
Validation	Acceptance testing performed within the project team.
Discussion	The current implementation supports page-level toggling via the config file. A more granular approach allowing activation of individual sub tabs would improve modularity and even satisfy user needs.

✓ Optional | FR7: OSPF Neighbor States Troubleshooting

Detail	The frontend (frr-mad-tui) offers a clear, focused display of OSPF neighbors, showing their states and adjacency details without clutter. Neighbor states are also included in the Router LSA (Type 1 LSA) Transit Network table, helping users determine if the local router, a direct neighbor, or another transit network device is the Designated Router (DR).
Status	Fully implemented
Validation	Acceptance test with stakeholder
Discussion	Enhancing the feature with state change detection and historical tracking would further improve troubleshooting capabilities.

✕ Optional | FR8: BGP Session States Troubleshooting

Detail	-
Status	Not implemented
Comment	BGP-related functionality was entirely excluded from the project scope, as discussed in FR1-2 evaluation.

✓ Optional | FR9: TUI History

Detail	All command inputs from the frontend (frr-mad-tui) client are logged to a dedicated log file. This applies to both the built-in vtysh shell and bash shell. Additional metadata including timestamps and source identifiers indicate when commands were executed and from which shell.
Status	Partially implemented.
Validation	Acceptance testing performed within the project team.
Reason (if not completed)	Not fully completed due to time constraints.
Discussion	The foundational infrastructure has been established. Interactions with frontend (frr-mad-tui) are already logged with sufficient metadata for proper categorization. Only an additional feature for input history lookup remains to be implemented.

✓ Optional | FR10: Issue Solving via TUI

Detail	The Custom Shell enables users to execute vtysh commands directly within the TUI, allowing manual resolution of routing anomalies. This feature supports issue-solving functionality, but must be used with care due to the constraint defined in NFR10, which requires the router configuration to remain unchanged between frr-mad-tui startup and shutdown.
Status	Partially implemented
Validation	Acceptance testing performed within the project group.
Reason (if not completed)	Not fully completed due to time constraints.
Discussion	While the current shell feature enables manual issue resolution, a major improvement would be a preview mechanism that estimates the impact of a configuration change. Such a feature could simulate the resulting OSPF state based on current neighbors and received/sent LSAs. However, implementing this feature would require significant development effort and depends on other optional functionalities that exceed the time constraints of this thesis.

✕ Optional | FR11: Guided Fixes for Misadvertised Routes

Detail	This feature was planned as an extension of the previous FR (FR10). For further details, see FR11.
Status	Not implemented
Reason (if not completed)	Not implemented due to time constraints and the unexpectedly large scope of work.
Discussion	This feature has the potential to provide significant support for the industry partner's mission control operator. The steps outlined in Industry Partner Specific Problems could potentially be executed with a single action.

✓ Optional | FR12: Export Routing Anomaly Analysis Results

Detail	The frontend (frr-mad-tui) implements the feature "Export Options," accessible from every page that displays backend (frr-mad-analyzer) data. It allows users to export any parsed data, such as OSPF metrics, routing tables, or anomaly results directly to the clipboard or to a temporary file. The export path can be configured via the application's configuration file.
Status	Fully implemented
Validation	Acceptance testing with stakeholder
Discussion	The only reliable way to transmit a "copy to clipboard" command from a Docker container running on a VM (accessed via SSH) to the user's local terminal is through the OSC52 Protocol. For example, in iTerm2, this feature can be enabled under <i>Settings > General > Selection > Access: <allow></i> . This feature includes enhancements beyond the original specification.

✕ Optional | FR13: Knowledge Database for Manual Fixes

Detail	This feature was proposed spontaneously during a meeting but remains unimplemented due to its broad scope and lack of detailed planning.
Status	Not implemented
Reason (if not completed)	Not implemented due to time constraints and the unexpectedly large scope of work.
Discussion	Such a feature would help novice users by giving them access to insights from more experienced community members.

 MVP | **FR14:** FRRMon Replacement

Detail	The system completely replaces FRRMon's export functionality while adding anomaly detection and interactive TUI capabilities. It maintains Prometheus compatibility while providing more comprehensive OSPF-specific checks than the original tool.
Status	Fully implemented
Validation	Successfully validated by the stakeholder through testing on their production systems, confirming all monitoring workflows work as expected.
Discussion	The solution successfully replaced FRRMon while offering superior capabilities. The main improvement is the interactive TUI interface for immediate troubleshooting. While more resource-intensive than FRRMon, the benefits of real-time analysis justify this trade-off. Future enhancements could include direct Prometheus export.

 Optional | **FR15:** Dynamic Route Filtering in TUI

Detail	The frr-mad-tui provides dynamic filtering capabilities across all pages, allowing users to interactively filter routes or LSA contents during a session. The Filter is one of the core usability features, significantly improving readability, especially in large-scale routing environments. Filters can be stored temporarily and re-applied across views, aligning with the design goals of FR15.
Status	Fully implemented
Validation	Acceptance testing with stakeholder
Discussion	This is already a highly practical feature within the FRR-MAD application. However, its utility could be further improved by extending the filter to support regular expression (regex) searches.

✓ **Optional** | **FR16:** Export Detected Anomalies

Detail	The system fully implements anomaly export functionality to Prometheus, covering all detected OSPF anomalies including overadvertised and unadvertised routes. Metrics are categorized by source (Router, External, NSSA External, RIB-to-FIB, and LSDB-to-RIB) and include detailed contextual information about each anomaly. The exporter runs at a configurable interval with automatic retry logic for failed updates.
Status	Fully implemented
Validation	Unit Tests, Acceptance Test with stakeholder
Discussion	The implementation provides comprehensive anomaly reporting with detailed labels for effective monitoring. The modular design allows for easy extension to additional anomaly types. The retry mechanism ensures reliability during temporary failures. Future enhancements could include additional diagnostic metadata in the exported metrics.

3.1.2 Non-Functional Requirements Evaluation

- ✓ **NFR1:** Presentation of information dashboard extension
Displays OSPF and system information in a dashboard view.
- ✓ **NFR2:** Tab selection
Allows switching between TUI tabs via click or shortcut.
- ✗ **NFR3:** Presentation of information BGP extension
Extends the dashboard to display BGP information.
- ✓ **NFR4:** Correctness of `frr_exporter` metrics
Ensures the `frr_exporter` detects and exports anomalies within 120 seconds.
- ✓ **NFR5-1:** TUI read mode only
Ensures the TUI operates in read-only mode by default.
- ✓ **NFR5-2:** TUI read/write mode
Allows switching between read-only and read/write modes with visual cues.
- ✓ **NFR6:** Limited exported metrics
Ensures the `frr_exporter` only exports OSPF- and BGP-related metrics.

- ✓ **NFR7: TUI User Experience**
Ensures the TUI uses color schemes for warnings and informational messages.
- ✓ **NFR8: TUI Resolution Support**
Ensures the TUI displays correctly at 1280×720 resolution.
- ✓ **NFR9: Lazy Loading of TUI**
Ensures the TUI only loads content for the currently selected tab.
- ✓ **NFR10: Integration of TUI in Dev/Prod Environments**
Ensures the TUI can be easily integrated into Linux environments.
- ✓ **NFR11: Guided or Automated Implementation of TUI**
Provides a script to automate TUI installation in five commands.
- ✓ **NFR12: Testing Environment for frr_exporter and TUI**
Ensures a test environment is provided for both frr_exporter and the TUI.

3.2 Future Directions

The FRR-MAD project successfully delivered its core OSPF monitoring and anomaly detection capabilities, but several areas remain for future development. Based on implementation challenges and stakeholder feedback, we identify four key improvement vectors:

3.2.1 Protocol Support Extensions

- **BGP integration:** Implement monitoring and anomaly detection for BGP (FR1-2, FR5, FR8) to create comprehensive routing visibility
- **FRR version compatibility:** Refactor the JSON parser to support newer FRR releases (post-8.5) as identified in FR2 evaluation

3.2.2 Anomaly Detection Enhancements

- **Guided remediation:** Develop the planned guided fixes system (FR11) with configuration impact simulation (FR10)
- **LSA coverage expansion:** Extend detection to Network (Type 2) and Summary (Type 3/4) LSAs (FR3 evaluation)

- **Interface generalization:** Refactor interface parsing to correctly handle /32 point-to-point configurations and avoid false positives caused by current industry-specific adjustments (Problem 2)

3.2.3 User Experience Improvements

- **Intuitive shell tab:** Enhance the TUI shell with command autocompletion, history navigation, and syntax highlighting to improve interactive usage
- **Advanced filtering:** Implement regular expression support for dynamic filters (FR15)

3.2.4 Community and Operational Features

- **Knowledge base:** Implement the proposed troubleshooting database (FR13)
- **Enhanced metrics:** Expand Prometheus exports with diagnostic metadata (FR16)

This roadmap prioritizes features that would most significantly enhance operational utility while maintaining the system's modular architecture. The proposed extensions address both immediate technical gaps and long-term usability requirements identified during the project.

Bibliography

- [1] Anarkiwi. google-quagga: Google fork of quagga routing software. <https://github.com/anarkiwi/google-quagga>, 2024. Last accessed: 2024-03-02.
- [2] arc42 Team. arc42: Effective, lean and pragmatic architecture documentation and communication. <https://arc42.org/documentation/>. Template and methodology for software architecture documentation with 12 standardized sections; accessed June 12, 2025.
- [3] BaiMeow. Networkmonitor. <https://github.com/BaiMeow/NetworkMonitor>, 2024. Last accessed: 2024-03-02.
- [4] charmbracelet/bubbletea. Bubble tea: A powerful little tui framework. <https://github.com/charmbracelet/bubbletea>. Go framework for building terminal applications based on The Elm Architecture; accessed June 12, 2025.
- [5] charmbracelet/lipgloss. Lip gloss: Style definitions for nice terminal layouts. <https://github.com/charmbracelet/lipgloss>. Go library for styling terminal applications with CSS-like syntax, tables, lists, and trees; accessed June 12, 2025.
- [6] Cisco Systems. Open shortest path first (ospf) administrative distance. <https://www.cisco.com/c/en/us/support/docs/ip/border-gateway-protocol-bgp/15986-admin-distance.html#toc-hId--1692530466>. Explains AD values such as 110 for OSPF; updated Sept 27, 2024; accessed June 10, 2025.
- [7] Claude. Claude: Conversational ai model and image generator. <https://claude.ai/>, 2024. Last accessed: 2024-12-19.
- [8] Practical DevSecOps. Threat modeling vs risk assessment: Understanding the difference. <https://www.practical-devsecops.com/threat-modeling-vs-risk-assessment/>, 2024. Last accessed: 2024-11-18.

- [9] Roman Dodin and Srlabs Team. Containerlab: Container-based networking lab. <https://containerlab.dev/>, 2024. Active open-source project (latest release: v0.68.0).
- [10] Inc. Figma. Figma: Collaborative interface design tool. <https://www.figma.com>, 2024. Last accessed: 2024-11-25.
- [11] Google Protocol Buffers Team. Protocol buffer basics: Go. <https://protobuf.dev/getting-started/gotutorial/>. Tutorial for using Protocol Buffers with Go, covering message definition, compilation, and API usage; accessed June 12, 2025.
- [12] Craig Larman. Applying evolutionary requirements. https://www.craiglarman.com/wiki/downloads/applying_uml/larman-ch5-applying-evolutionary-requirements.pdf, 2002. Accessed: 2025-03-12.
- [13] Craig Larman. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development (3rd Edition)*. Prentice Hall PTR, Upper Saddle River, NJ, USA, 2004.
- [14] JGraph Ltd and draw.io AG. diagrams.net (formerly draw.io): Online drawing tool. <https://app.diagrams.net/>, 2024. Last accessed: 2024-12-19.
- [15] M. Mitchell, J. Dickinson, and G. Huston. Rfc 6996: Autonomous system (as) reservation for private use. <https://datatracker.ietf.org/doc/rfc6996>, 2013.
- [16] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Robert C. Martin Series. Prentice Hall, 2009.
- [17] Livio Mauchle, Simon Hefti, Martyn Foreman, Yannick Städeli, and Luca Köppel. Swiss-cardgames: Final documentation. Technical report, School of Computer Science, OST Eastern Switzerland University of Applied Sciences, 2024.
- [18] Nvidia Networking. Cumulus linux 4.1 - frouting overview. <https://docs.nvidia.com/networking-ethernet-software/cumulus-linux-41/Layer-3/FRRouting-Overview/>, 2024. Last accessed: 2024-03-02.
- [19] Juniper Networks. Juniper, configure ospf route control. <https://www.juniper.net/documentation/us/en/software/junos/ospf/topics/topic-map/configuring-ospf-route-control1.html>. Last accessed: 2025-05-30.
- [20] Olaf Zimmermann. Domain-driven design. Lecture: Application Architecture (AppArch), OST - Eastern Switzerland University of Applied Sciences. Course materials available at <https://ozimmer.ch/>; accessed June 12, 2025.
- [21] OpenAI. Chatgpt: Conversational ai model and image generator. <https://openai.com/chatgpt>, 2024. Last accessed: 2024-12-19.
- [22] OSRG. Gobgp: Open source bgp implementation. <https://github.com/osrg/gobgp>, 2024. Last accessed: 2024-03-02.

- [23] Batfish Project. Batfish: Network configuration analysis tool. <https://github.com/batfish/batfish>, 2024. Last accessed: 2024-03-02.
- [24] Simon Brown. The c4 model for visualising software architecture. <https://c4model.com/>. Hierarchical approach to software architecture diagramming with context, containers, components, and code levels; accessed June 12, 2025.
- [25] SNAS. Openbmp: Bgp monitoring protocol collector. <https://github.com/SNAS/openbmp>, 2024. Last accessed: 2024-03-02.
- [26] SoftwareTestingHelp. Top 15 network monitoring tools. <https://www.softwaretestinghelp.com/network-monitoring-tools/>, 2024. Last accessed: 2024-03-02.
- [27] spf13/cobra. Cobra: A commander for modern go cli interactions. <https://github.com/spf13/cobra>. Go library for creating powerful modern CLI applications with subcommands, flags, and auto-completion; accessed June 12, 2025.
- [28] spf13/viper. Viper: Go configuration with fangs. <https://github.com/spf13/viper>. Go configuration library supporting JSON, TOML, YAML, HCL, INI, envfile and Java properties formats; accessed June 12, 2025.
- [29] Tynany. frr_exporter: Prometheus exporter for frrouting. https://github.com/tynany/frr_exporter, 2024. Last accessed: 2024-03-02.
- [30] Vadims06. Ospf topology watcher. <https://github.com/Vadims06/ospfwatcher>, 2024. Last accessed: 2024-03-02.

List of Figures

1	Simplified Architecture	iii
2	frr-mad-tui Anomaly Dashboard	iv
1.1	Design of frr-mad-analyzer Components	8
1.2	frr-mad-tui OSPF Dashboard	9
2.1	frr-mad-tui Info Page	14
2.2	frr-mad-tui Export Page	17
2.3	frr-mad-tui Anomaly Dashboard	20
2.4	frr-mad-tui Filter Function	21
2.5	frr-mad-tui vtysh input	22
2.6	Development and Test Environment Overview	23
2.7	High Level Abstraction Architecture	24
2.8	Frontend to FRR Communication	25
2.9	Frontend to Backend Communication	26
2.10	Backend internal communication	27
2.11	Backend - Prometheus Communication	28
2.12	Workflow Flowchart	36
2.13	Elaboration Result: Initial Architecture	37
2.14	Development Phase: Adjusted Architecture	38
A.1	Containerlab - Network Concept 1	63
A.2	Containerlab - Network Concept 2	64

List of Tables

1.1	Routing Issues between Static Config and LSDB	6
2.1	Supported FRR Data Types	12
2.2	Purpose of each fr-mad-tui Page	14
2.3	Comparison of isState and shouldState	18
2.4	Examples of Route Advertisement Issues	19
2.5	Comparison of Route Metrics and Anomalies Features	19
2.7	Link Types based on Interface State	39
2.9	Difference of Router LSA (Type 1 LSA) from Numbered/Unnumbered Interfaces	40

List of Algorithms

1	Convert OSPF Link States to Network Addresses Map	29
2	Detect Unadvertised Networks	29
3	Detect Overadvertised Networks	30
4	Unix Socket Message Handler	31
5	Generic Service Handler	32
6	Metric Export Procedure	33
7	Anomaly Export Procedure	34

Listings

1.1	Example Router Interface Config	5
2.1	Adjust Export Path in Configuration File	17
2.2	Frontend Structure	34
A.1	Announce Prefix as Type 5	60
A.2	Announce Prefix as Type 1	60
A.3	Example Access-List	61
A.4	Build Instructions	64
A.5	Build Instructions with Build Flag Override	65
A.6	Example Configuration File	65
A.7	frr-mad-analyzer help menu	66
A.8	frr-mad-analyzer command help	67

Chapter A

Miscellaneous

A.1 Industry Partner's Anomaly Occurrence

When reconfiguring a specific interface, an unforeseen anomaly occurs. The goal is, that a prefix would be advertised as a stub network within a Router LSA (Type 1 LSA), rather than as an AS External LSA (Type 5 LSA). One can observe this change by comparing the initial configuration (Announce Prefix as Type 5) with the adjusted configuration (Announce Prefix as Type 1), specifically by examining the configuration section for interface eth1.

```
1 ! Initial Configuration:
2 !
3 ip route 10.0.0.0/17 10.0.0.255
4 !
5 interface eth1
6     ip address 10.0.0.0/27
7 exit
8 !
9 router ospf
10     ospf router-id 100.100.100.100
11     redistribute static route-map lanroutes metric-type 1
12     redistribute connected route-map lanroutes metric-type 1
13 exit
14 !
```

Listing A.1: Announce Prefix as Type 5

```
1 ! Adjusted Configuration:
2 !
3 ip route 10.0.0.0/17 10.0.0.255
4 !
```

```
5 interface eth1
6     ip address 10.0.0.1/27
7     ip ospf passive
8     ip ospf area 0.0.0.0
9 exit
10 !
11 router ospf
12     ospf router-id 100.100.100.100
13     redistribute static route-map lanroutes metric-type 1
14 exit
15 !
```

Listing A.2: Announce Prefix as Type 1

To maintain clarity between the two configuration examples, the route-map is omitted here, but a possible version might look as follows:

```
1 !
2 access-list denyall seq 5 deny any
3 access-list localsite seq 10 permit 10.0.0.0/17
4 access-list localsite seq 15 permit 10.0.0.0/27
5 !
6 route-map lanroutes permit 10
7     match ip address localsite
8 exit
9 !
```

Listing A.3: Example Access-List

However, while implementing this change, a specific error occurred. The following enumeration outlines the issue in detail:

1. config is changed according to code snippet above
2. stub network is announced correctly
3. Anomaly: static route 10.0.0.0/17 is missing
4. Solution Step 1: remove and add the access-list entry 10.0.0.0/17
5. Anomaly: static route 10.0.0.0/27 is advertised and 10.0.0.0/17 still missing
6. Solution Step 2:
 - (a) remove correct (10.0.0.0/17) network from access list
 - (b) add wrong (10.0.0.0/27) network to access list
 - (c) remove wrong (10.0.0.0/27) network from access list
 - (d) add correct (10.0.0.0/17) network to access list

A.2 Development and Test Environment Architecture

AS described in the RFC 6996, private use ASNs were used [\[15\]](#).

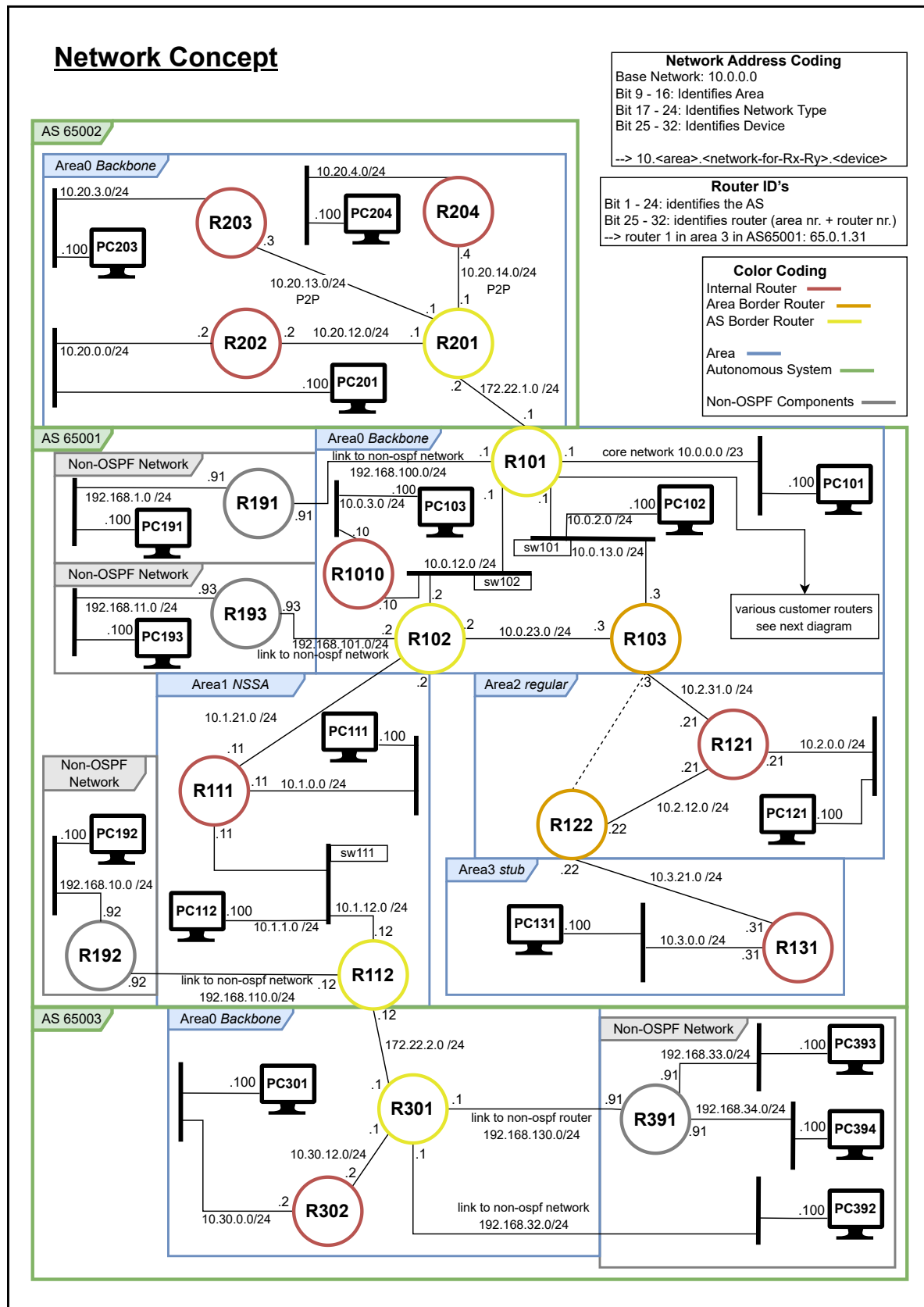


Figure A.1: Containerlab - Network Concept 1

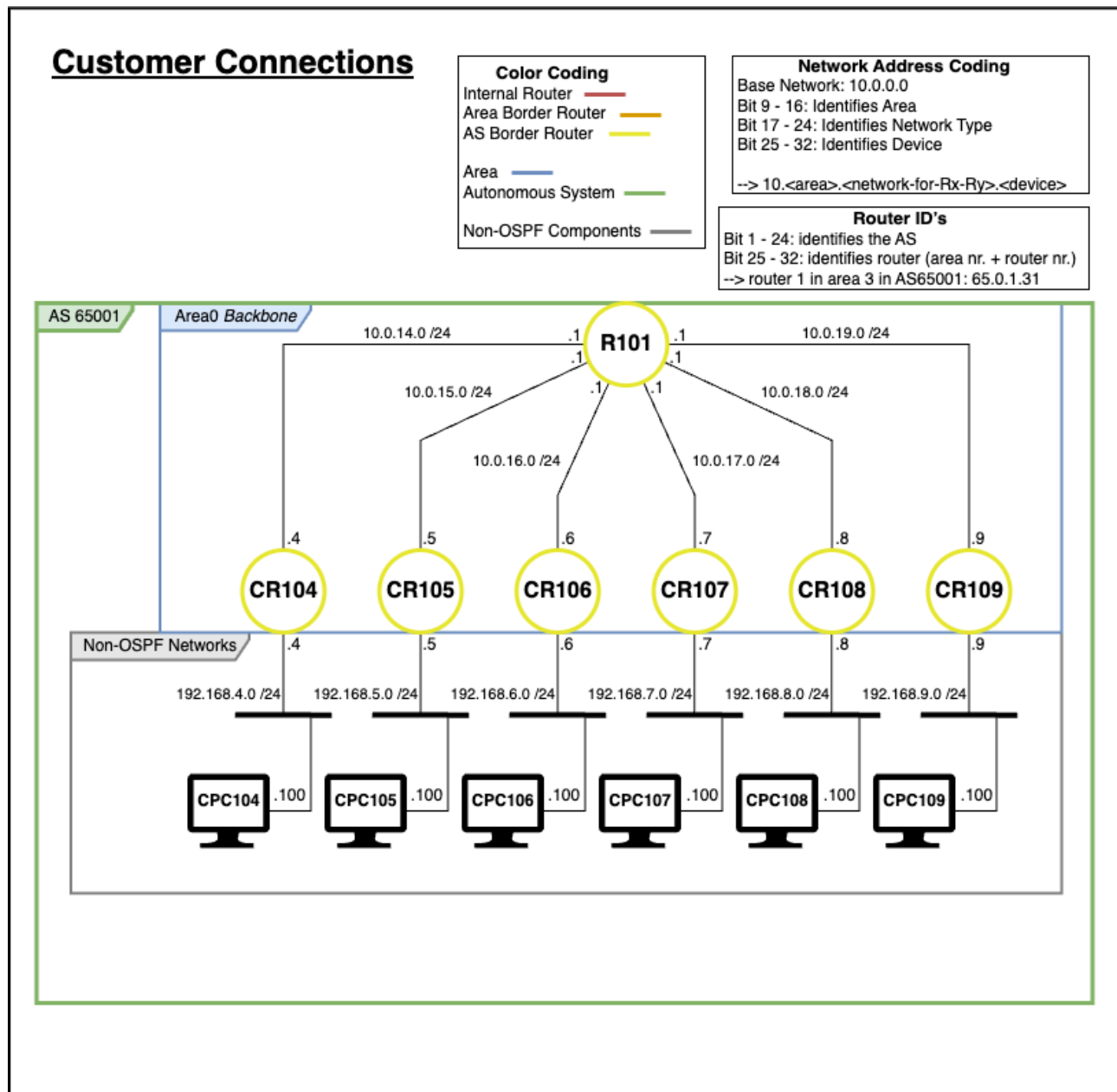


Figure A.2: Containerlab - Network Concept 2

A.3 Setup and Installation

In this subsection the FRR-MAD application installation and utilization will be outlined. Similar instructions are provided in GitHub [README](#).

A.3.1 Build Instructions

Currently, no pre-built packages are available, requiring the project to be built before use. The applications should be built statically to eliminate dependency issues. The default version can be built by executing make, though this imposes restrictions on configuration path locations.

```

1 mkdir -p /tmp/frr-mad-binaries/
2 cd src/backend && CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build
  -ldflags='-s' -o /tmp/frr-mad-binaries/frr-mad-analyzer ./cmd/frr-analyzer
3 cd ../../
4 cd src/frontend && CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build
  -ldflags='-s' -o /tmp/frr-mad-binaries/frr-mad-tui ./cmd/tui
5 cd ../../

```

Listing A.4: Build Instructions

The default configuration path can be overridden during the build process using build flags. When the correct build flags (e.g., `-X configs.ConfigLocation=/path/to/configuration.yaml`) are specified, the built application reflects these changes. This is particularly valuable when the default configuration path does not conform to company policies or other requirements.

```

1 mkdir -p /tmp/frr-mad-binaries/
2 cd src/backend && CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -ldflags='-s
  -X configs.ConfigLocation=/path/to/configuration.yaml' -o
  /tmp/frr-mad-binaries/frr-mad-analyzer ./cmd/frr-analyzer
3 cd ../../
4 cd src/frontend && CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build
  -ldflags='-s -X configs.ConfigLocation=/path/to/configuration.yaml' -o
  /tmp/frr-mad-binaries/frr-mad-tui ./cmd/tui
5 cd ../../

```

Listing A.5: Build Instructions with Build Flag Override

Once the application is built, a configuration file must be provided. An example configuration scheme is available [here](#). All values are required, and if this application runs with restricted permissions, access to the specified locations and files must be granted.

```

1 default:
2   tempfiles: /tmp/frr-mad
3   exportpath: /tmp/frr-mad/exports
4   logpath: /var/log/frr-mad
5   # default is info
6   debuglevel: error
7
8 frrmadtui:
9   pages:
10    ospf:
11      enabled: true
12    rib:
13      enabled: true
14    shell:
15      enabled: true
16
17 socket:

```



```
18  unixsocketlocation: /var/run/frr-mad
19  unixsocketname: analyzer.sock
20  sockettype: unix
21
22 aggregator:
23   frrconfigpath: /etc/frr/frr.conf
24   pollinterval: 5
25   socketpath: /var/run/frr
26
27 exporter:
28   # default: Port: 9091
29   OSPFRouterData: false
30   OSPFNetworkData: false
31   OSPFSummaryData: false
32   OSPFAsbrSummaryData: false
33   OSPFExternalData: false
34   OSPFNssaExternalData: false
35   OSPFDatabase: false
36   OSPFNeighbors: false
37   InterfaceList: false
38   RouteList: false
```

Listing A.6: Example Configuration File

A.3.2 Application Launch

After completing the build process, the build folder will contain two files:

frr-mad-tui and frr-mad-analyzer.

As previously explained, frr-mad-analyzer operates as a daemon, while frr-mad-tui functions as its client. The frr-mad-tui cannot operate without the frr-mad-analyzer daemon running. To start frr-mad-analyzer executing `frr-mad-analyzer start` is enough. This spawns a new child process that runs in the background. According to the configuration, a Unix Domain Socket will be created, different files touched and a network socket opened. The locations can be adjusted in said configuration.

For detailed information about the tool's capabilities, executing it without arguments displays a help menu.

```
1 $> frr-mad-analyzer
2 A CLI tool for managing the FRR-MAD application.
3
4 Usage:
5   frr-mad-analyzer [command]
6
7 Available Commands:
8   completion  Generate the autocompletion script for the specified shell
9   help        Help about any command
```

```

10 restart      Restart the FRR-MAD application
11 start        Start the FRR-MAD application
12 stop         Stop the FRR-MAD application
13 version      show version number and exit
14
15 Flags:
16   -h, --help  help for frr-mad-analyzer
17
18 Use "frr-mad-analyzer [command] --help" for more information about a command.

```

Listing A.7: frr-mad-analyzer help menu

Some commands provide extended options. While metric exports can be configured through the configuration file, the start command also accepts additional arguments. This is particularly useful during testing phases.

```

1 $> frr-mad-analyzer help start
2 Start the FRR-MAD application
3
4 Usage:
5   frr-mad-analyzer start [flags]
6
7 Flags:
8   -c, --configFile string    Provide path overwriting default configuration
                               file location.
9   -h, --help                 help for start
10  --interface-list            Enable interface list metrics
11  --ospf-asbr-summary         Enable OSPF ASBR summary metrics
12  --ospf-database             Enable OSPF database metrics
13  --ospf-external             Enable OSPF external route metrics
14  --ospf-neighbors            Enable OSPF neighbor metrics
15  --ospf-network              Enable OSPF network metrics
16  --ospf-nssa-external        Enable OSPF NSSA external route metrics
17  --ospf-router               Enable OSPF router metrics
18  --ospf-summary              Enable OSPF summary metrics
19  --route-list                Enable route list metrics

```

Listing A.8: frr-mad-analyzer command help

Once the daemon is running, starting frr-mad-tui is straightforward—simply execute the binary. Note that frr-mad-tui requires the configuration file location to be defined. Three methods are available:

- Place the configuration file at the default location: `/etc/frr-mad/main.yaml`
- Use build flags during the build process to specify an alternative configuration file location.
- Export an environment variable: `export FRR_MAD_CONFFILE=/path/to/configuration` with the configuration file path.